Absolute Return Forecasting: From Linear Models to Recurrent Neural Networks

Edouard Blanc Clément Dardare Julien Simeoni

Machine Learning For Finance

March 30, 2026

Contents

1 Introduction **3**
1.1 Motivation . . . 3
1.2 Objective . . . 3
1.3 Dataset . . . 3
1.4 Evaluation . . . 3
1.5 Paper Structure . . . 4

2 Data **5**
2.1 Dataset Construction . . . 5
2.2 Descriptive Statistics . . . 5
2.3 Log-Return Time Series . . . 6
2.4 Realized Volatility and Regime Identification . . . 6
2.5 Autocorrelation Analysis . . . 7
2.6 Feature Correlation Structure . . . 8
2.7 Summary . . . 8

3 Linear Regression Baseline **9**
3.1 Standard Linear Regression . . . 9
3.1.1 Setup . . . 9
3.1.2 Results . . . 9
3.1.3 Coefficient Analysis . . . 9
3.1.4 Predicted vs. Actual . . . 10
3.2 Feature Engineering . . . 10
3.3 Regime Analysis . . . 11
3.4 GARCH(1,1) Baseline . . . 11
3.4.1 Estimated Parameters . . . 11
3.4.2 Regime Performance . . . 12
3.5 Summary of Limitations . . . 12

4 Feedforward Neural Network **13**
4.1 Model and Validation . . . 13
4.2 Hyperparameter Tuning . . . 13
4.2.1 Activation Function . . . 13
4.2.2 Weight Initialisation . . . 14
4.2.3 Optimizer and Momentum . . . 16
4.2.4 Temporal Window . . . 18
4.3 Diagnostics: Inside the Trained Network . . . 19
4.3.1 Weight and Activation Distributions . . . 20
4.3.2 Gradient Flow . . . 22
4.3.3 Weight Evolution During Training . . . 22
4.4 Structural Limitations . . . 23
4.4.1 Order Blindness . . . 23
4.4.2 Regime Blindness . . . 24
4.5 Summary . . . 24

5 Recurrent Neural Network **26**
5.1 From FNN to RNN . . . 26
5.2 Architecture and Training Setup . . . 26
5.3 Baseline Results . . . 27
5.4 Diagnostics: What Did the Model Learn? . . . 30
5.4.1 Weight Distributions . . . 30
5.4.2 Hidden State Activity . . . 31
5.4.3 Input Sensitivity by Lag . . . 32
5.5 Failure Analysis . . . 33
5.5.1 Residuals Over Time . . . 33
5.5.2 Error vs Magnitude . . . 34
5.5.3 Worst Predictions Per Regime . . . 34
5.6 Vanishing Gradient Diagnosis . . . 36
5.6.1 W_{hh} Spectral Radius . . . 36
5.6.2 Output Sensitivity per Time Step . . . 37
5.6.3 Effective Memory . . . 37
5.6.4 Effective Memory by Regime . . . 38
5.7 Hyperparameter Grid Search . . . 39
5.8 Summary . . . 39

6 Conclusion **41**
6.1 Cross-Model Comparison . . . 41
6.2 Key Findings . . . 41
6.3 Limitations and Future Work . . . 42

1 Introduction

1.1 Motivation

Accurately forecasting the magnitude of future asset returns is a central problem in quantitative finance. While the *direction* of price changes is notoriously difficult to predict, the *amplitude* of those changes (often proxied by realized volatility) exhibits strong and exploitable serial dependence. This phenomenon is characterised by *volatility clustering*: large price moves tend to be followed by large moves, and calm periods persist until a new shock arrives.

Predicting the absolute return of the S&P 500 index therefore serves a dual purpose: it is both a well-posed statistical problem with a meaningful signal in the data and a practically relevant task for risk management, option pricing, and portfolio construction.

1.2 Objective

This project empirically compares a progression of models of increasing expressive power applied to the same forecasting task:

1. **Linear Regression** — a transparent but structurally constrained baseline.
2. **GARCH(1,1)** — the industry-standard parametric volatility model.
3. **Feedforward Neural Network (FNN)** — a non-linear model that treats the input as a flat, orderless feature vector.
4. **Recurrent Neural Network (RNN)** — a sequential model that can exploit the temporal ordering of lags.

Rather than searching for the best possible model, the goal is to *understand the limitations of each architecture* and to show how each successive model overcomes (some of) the shortcomings of its predecessor.

1.3 Dataset

The dataset is built from the daily log-returns of the SPY ETF (S&P 500 tracker), downloaded via the `yfinance` library. The sample spans **February 1993 to March 2026**, yielding **8 343 daily observations**.

The forecasting problem is formulated as a sliding-window regression:

- **Input** X_t: the L most recent daily log-returns $(r_{t-1}, r_{t-2}, \ldots, r_{t-L})$, with $L \geq 10$.
- **Target** y_t: the realized absolute return on day t, computed as the rolling standard deviation of returns over a short window (a proxy for next-day volatility).

1.4 Evaluation

Models are evaluated using three metrics, computed on a held-out test set:

- **MAE** (Mean Absolute Error) — primary metric for cross-model comparison.
- **RMSE** (Root Mean Squared Error) — penalises large errors more heavily.

- **QLIKE** (Quasi-Likelihood loss) — a standard volatility-forecasting loss that penalises systematic under-prediction.

Because market conditions vary dramatically, all models are also evaluated separately on two **market regimes** identified from the data:

- **Calm regime** (≈90% of observations): low, stable volatility.
- **Crisis regime** (≈10% of observations): sharp spikes corresponding to the dot-com crash, the 2008 Global Financial Crisis, and the COVID shock.

1.5 Paper Structure

Section 2 describes the dataset and its key statistical properties. Section 3 presents the linear regression and GARCH baselines. Sections 4 and 5 introduce the feedforward and recurrent neural network architectures respectively. Section 6 closes the report with a cross-model comparison and a discussion of limitations and future directions.

2 Data

2.1 Dataset Construction

Daily log-returns for the SPY ETF are downloaded via `yfinance` and stored locally in `data/df_sp_500_log_ret.csv`. The raw return on day t is defined as

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right),$$

where P_t denotes the adjusted closing price. A sliding-window dataset is then constructed with a look-back of $L \geq 10$ lags:

$$\begin{aligned} X_t &= (r_{t-1},\, r_{t-2},\, \ldots,\, r_{t-L}) \in \mathbb{R}^L, \\ y_t &= \sigma_t \in \mathbb{R}_+, \end{aligned}$$

where σ_t is the realized absolute return (rolling standard deviation of returns over a short window), serving as a proxy for next-day volatility.

2.2 Descriptive Statistics

Table 1 reports summary statistics for the full sample.

Table 1: Descriptive statistics for S&P 500 daily log-returns (SPY, 1993–2026).

Statistic	Value
Observations	8 343
Start date	1993-02-01
End date	2026-03-24
Mean	0.0004
Std deviation	0.0117
Minimum	−0.1159
Maximum	+0.1356
Skewness	−0.248
Kurtosis	11.42

The distribution exhibits **negative skewness** (large negative shocks more common than large positive ones) and **excess kurtosis** of 11.42, confirming heavy tails relative to a Gaussian distribution. Both features are consistent with the well-known stylised facts of equity returns.

2.3 Log-Return Time Series

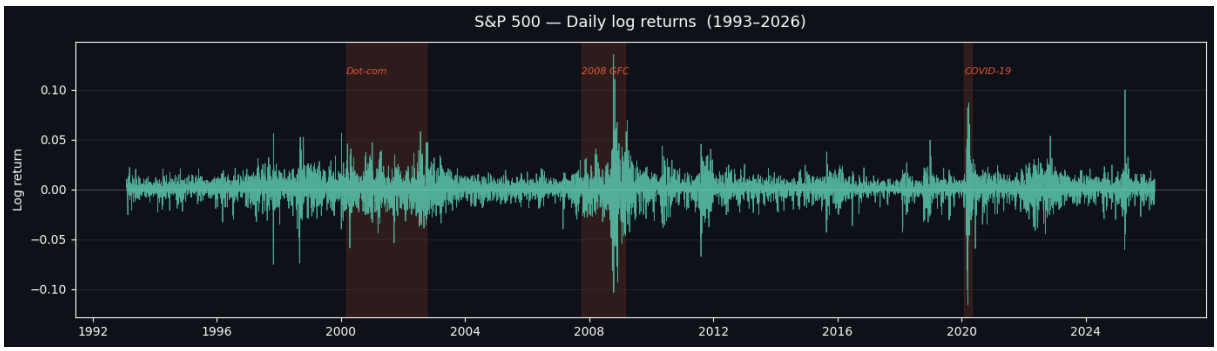

Figure 1: S&P 500 daily log-returns. Three major stress periods are clearly visible.

Returns oscillate around zero with no discernible trend, consistent with the efficient market hypothesis. Three major stress episodes are visible: the dot-com crash (2000–2002), the Global Financial Crisis (2008–2009), and the COVID shock (2020). These episodes exhibit the characteristic feature of **volatility clustering**: large moves arrive in bursts and calm periods are interspersed.

2.4 Realized Volatility and Regime Identification

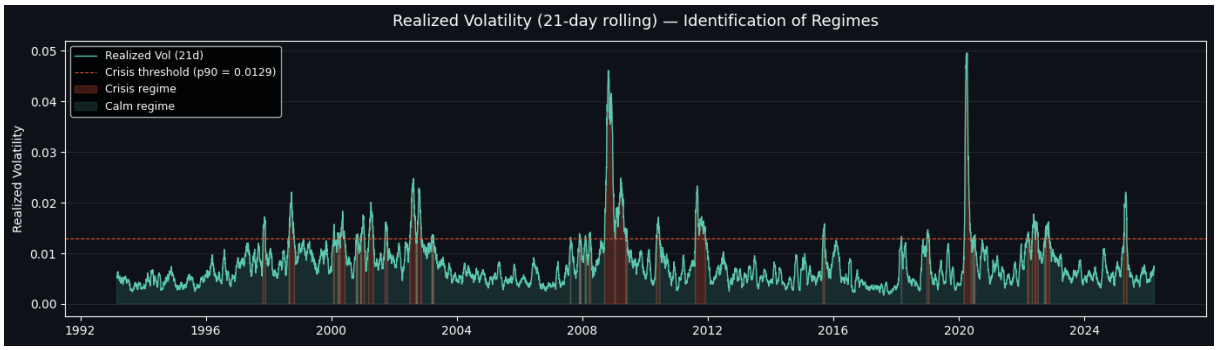

Figure 2: Realized absolute return (target variable y_t). The dashed line indicates the 90th-percentile threshold used to define the crisis regime.

A threshold at the **90th percentile** ($\sigma^* = 1.117$) is used to split the sample into two regimes:

Table 2: Market regime statistics.

Regime	Share of observations	Characterization
Calm	90%	Low, stable volatility
Crisis	10%	Sharp spikes; three major episodes

The pronounced asymmetry in regime frequency is critical: a model that performs well on average may still fail badly during the 10% of days that matter most for risk management.

2.5 Autocorrelation Analysis

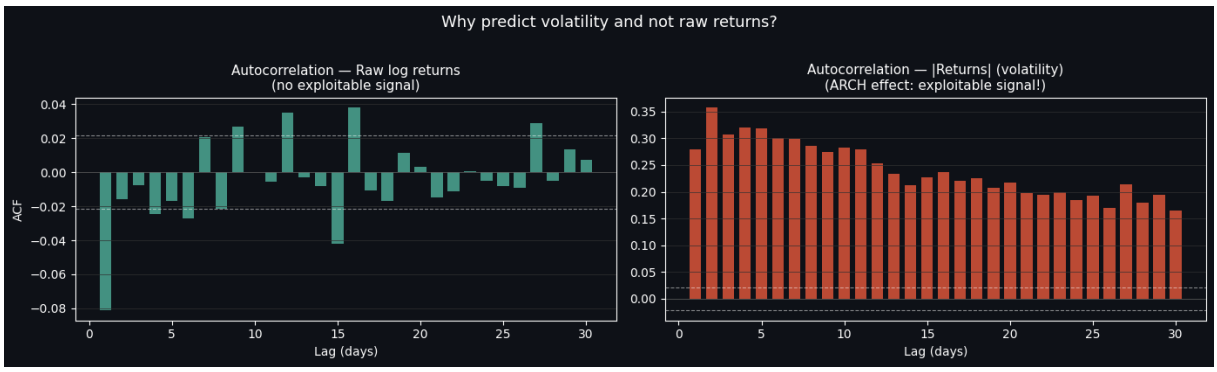

Figure 3: Autocorrelation of raw log-returns (near zero) vs absolute log-returns (strongly positive). The latter justifies using past absolute returns as features.

Two key findings:

- **Raw log-returns**: autocorrelation is essentially zero at all lags — the return series itself is unpredictable.
- **Absolute log-returns / realized volatility**: autocorrelation is strongly and persistently positive (the ARCH effect), justifying the use of past lagged returns as features for predicting future volatility.

2.6 Feature Correlation Structure

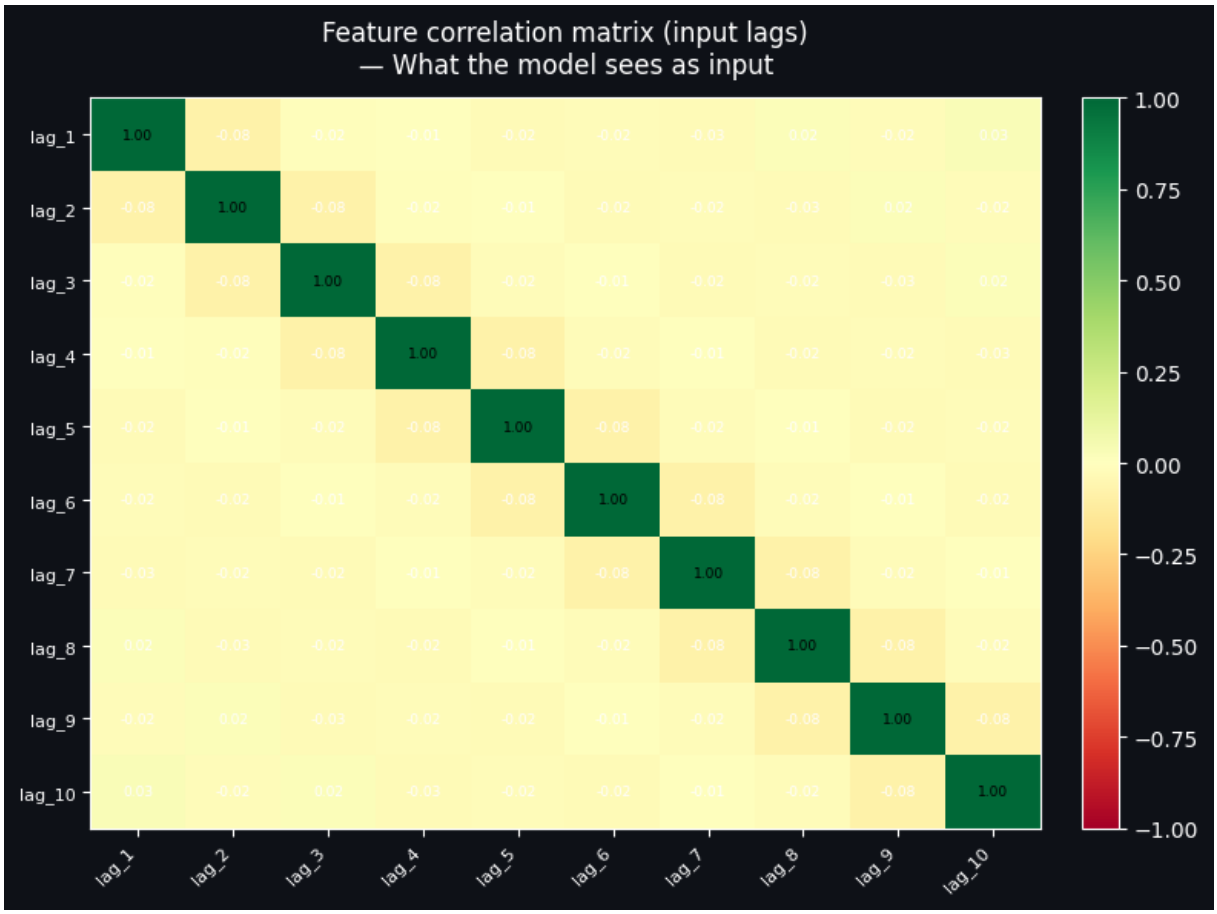

Figure 4: Pearson correlation matrix of the 10 lag features. Adjacent lags are positively correlated, reflecting the persistence of volatility regimes.

Adjacent lags are positively correlated due to volatility clustering: a large move on day $t-1$ is likely to be followed by a large move on day $t-2$. This introduces mild multicollinearity for linear models and gives sequential models such as the RNN a structural advantage.

2.7 Summary

Three statistical properties of the data guide the modelling choices throughout this report:

1. **Raw returns are unpredictable** (near-zero ACF) — the target must be the magnitude, not the sign, of returns.
2. **Volatility is autocorrelated** (ARCH effect) — past lags carry genuine predictive information.
3. **Two regimes coexist** — calm and crisis periods have fundamentally different statistical properties; a good model must handle both.

3 Linear Regression Baseline

This section establishes two classical baselines: ordinary least squares (OLS) regression and GARCH(1,1). Both are transparent and well-understood, making their failure modes informative for motivating the neural network architectures that follow.

3.1 Standard Linear Regression

3.1.1 Setup

An OLS model is trained on the 10 signed lag features with a standard train/test split. Features and target are scaled with a `StandardScaler` fitted on the training set only. The target $y_t = |r_t|$ is the realized absolute return, used throughout as a proxy for next-day volatility. The model takes the form

$$\hat{y}_t = \beta_0 + \sum_{k=1}^{10} \beta_k \, r_{t-k}.$$

3.1.2 Results

Table 3: Linear regression performance on the test set (scaled units).

Model	MAE	RMSE	QLIKE
Linear Regression (raw lags)	0.638	0.933	16.79
Linear Regression (engineered)	0.598	0.874	7.46

3.1.3 Coefficient Analysis

Table 4: OLS regression coefficients ranked by absolute magnitude.

Feature	Coefficient	$\lvert\beta_k\rvert$ rank
lag_9	-0.180	1
lag_8	-0.164	2
lag_10	-0.159	3
lag_7	-0.158	4
lag_5	-0.123	5
lag_6	-0.110	6
lag_3	-0.106	7
lag_1	-0.094	8
lag_4	-0.084	9
lag_2	-0.060	10

All coefficients are *negative* and small. This reveals the first fundamental limitation:

> **Symmetry blindness.** Linear regression on signed returns cannot treat $+x$ and $-x$ identically. Predicting volatility requires $f(x) \approx f(-x)$ (the absolute value function), which a linear model cannot represent. The model "gives up"

by assigning small negative coefficients and predicting near-zero values for all inputs.

3.1.4 Predicted vs. Actual

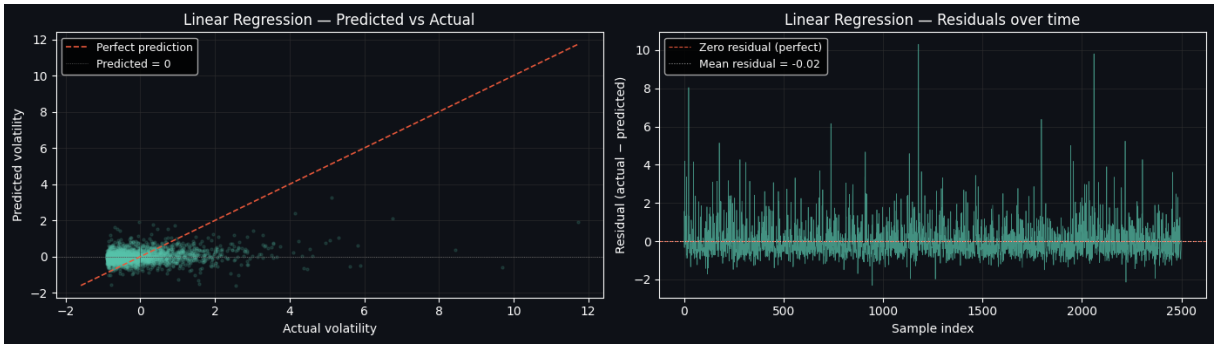

Figure 5: Linear regression: predicted vs. actual realized absolute return. The model systematically underpredicts during crisis episodes.

The model produces a near-constant prediction close to zero throughout the entire sample. Residuals are always positive and are dramatically amplified during crisis episodes.

3.2 Feature Engineering

Squaring the lag features forces the symmetry the model cannot learn from signed returns: $r_{t-k}^2 = |r_{t-k}|^2$ treats positive and negative shocks of equal magnitude identically. A *recency ratio* feature $r_{t-1}/\text{mean}(r_{t-2:t-5})$ is also added to capture short-term momentum in volatility.

Table 5: Impact of feature engineering on linear regression performance.

Features	MAE	RMSE	QLIKE
Raw signed lags	0.638	0.933	16.79
Squared lags + recency ratio	0.598	0.874	7.46
Improvement (Δ)	+6.3%	+6.4%	+55.6%

The QLIKE loss drops by 56%, showing that manual feature engineering substantially reduces systematic under-prediction. This foreshadows what the FNN can learn *automatically* from raw lags.

3.3 Regime Analysis

Table 6: Linear regression performance by market regime (raw lag features).

Regime	MAE	RMSE	QLIKE
Full	0.638	0.933	16.79
Calm	0.488	0.587	15.49
Crisis	1.987	2.367	28.46
Crisis / Calm ratio	4.1×	4.0×	1.8×

During crisis periods, MAE is **4× higher** than during calm periods. The model systematically underpredicts every volatility spike because it has no mechanism to detect a regime shift. This is the second structural limitation:

> **Regime blindness.** A single linear model fitted on the full sample cannot adapt its behaviour to different market conditions.

3.4 GARCH(1,1) Baseline

The GARCH(1,1) model provides a parametric volatility benchmark. The conditional variance follows:

$$\sigma_t^2 = \omega + \alpha\,\epsilon_{t-1}^2 + \beta\,\sigma_{t-1}^2,$$

where $\epsilon_t = r_t/\sigma_t$ are standardised residuals.

3.4.1 Estimated Parameters

Table 7: GARCH(1,1) maximum-likelihood parameter estimates.

Parameter	Estimate	Interpretation
ω (baseline variance)	0.0159	Long-run variance floor
α (shock sensitivity)	0.094	Reaction to new shocks
β (persistence)	0.895	Memory of past variance
$\alpha + \beta$ (total persistence)	0.989	Near unit-root variance process
Half-life of shock decay	60 days	Volatility shocks dissipate slowly

The near-unit-root persistence ($\alpha + \beta = 0.989$) confirms that volatility shocks take approximately **60 days** to decay to half their peak level.

3.4.2 Regime Performance

Table 8: GARCH(1,1) performance by market regime (unscaled returns).

Regime	MAE	RMSE	QLIKE
Full	0.00583	$0.000069^{\dagger}$	-3.98
Calm	0.00506	$0.000043^{\dagger}$	-4.17
Crisis	0.01281	$0.000296^{\dagger}$	-2.27
Crisis / Calm ratio	2.5×	6.9×	—

† These are MSE values; RMSE= $\sqrt{\text{MSE}}$.

GARCH improves on linear regression during calm periods but exhibits the same pattern of regime degradation: crisis MAE is 2.5× higher than calm MAE. Despite its elegant theoretical motivation, GARCH suffers from the third structural limitation:

> **Fixed functional form.** GARCH assumes a specific parametric update rule for variance. It cannot detect structural breaks or non-linear regime transitions.

3.5 Summary of Limitations

Table 9: Comparison of linear baseline models (MAE, full sample).

Model	MAE (full)	MAE (crisis)
Linear Regression (raw lags)	0.638	1.987
Linear Regression (engineered)	0.598	—
GARCH(1,1)	0.00583	0.01281

Three structural limitations motivate the turn to neural networks:

1. **Symmetry blindness** (linear regression on signed inputs cannot represent $|x|$).
2. **Regime blindness** (no model adapts its behaviour during crises).
3. **Fixed functional form** (GARCH assumes a pre-specified update rule).

A feedforward neural network with non-linear activations directly addresses limitation 1, and deeper architectures are expected to partially address limitations 2 and 3.

4 Feedforward Neural Network

4.1 Model and Validation

Architecture

The baseline FNN maps the L most recent signed log-returns to a scalar one-day-ahead volatility forecast through two hidden layers:

$$\mathbf{x} \in \mathbb{R}^L \xrightarrow{\text{Dense}_{32}} \mathbb{R}^{32} \xrightarrow{\text{Dense}_{16}} \mathbb{R}^{16} \xrightarrow{\text{Dense}_1} \hat{y} \in \mathbb{R}_+.$$

Both hidden layers use **ReLU** activations and **He normal** initialisation; the output neuron is linear. This choice of activation directly addresses the *symmetry blindness* of the linear baseline: two composed ReLU layers can approximate $|x|$ from signed inputs, recovering the signal in r_t without manual feature engineering.

Implementation validation

The custom NumPy framework (`dl_utils`) is cross-checked against an equivalent PyTorch implementation before any tuning.

Table 10: Custom NumPy implementation vs. PyTorch reference and linear baseline (lag = 10).

Implementation	MAE	Note
Custom `dl_utils` (NumPy)	0.6046	dropout = 0.5 on layer 1
PyTorch reference	0.5850	no dropout
Linear regression baseline	0.6377	

Both neural implementations beat the linear baseline. The small gap between them stems from the dropout regularisation applied only in the NumPy model: dropout slows convergence, raising the early-stopped MAE slightly.

4.2 Hyperparameter Tuning

All experiments in this section share the same default training protocol unless stated otherwise: MSE loss, batch size 32, maximum 300 epochs, early stopping with patience = 10 evaluated every 5 epochs.

4.2.1 Activation Function

Four activations are compared on the 32→16→1 architecture with plain SGD ($\alpha = 10^{-3}$, learning rate).

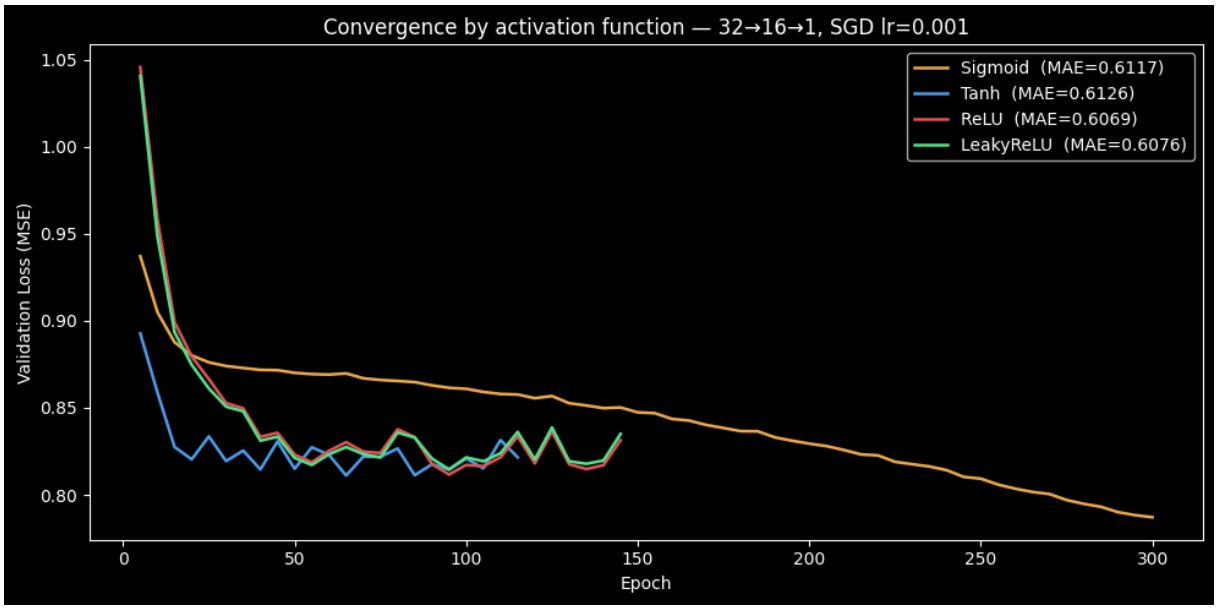

Figure 6: Validation loss (MSE) over 300 epochs for four activations. ReLU (red, MAE = 0.607) reaches the best test MAE; Sigmoid (orange) is still declining at epoch 300.

- **Sigmoid** (MAE 0.612): gradient saturation dramatically slows learning; the validation loss has not plateaued by epoch 300. The model does ultimately converge at this shallow depth after `StandardScaler` normalisation, but requires far more epochs than the others.
- **Tanh** (MAE 0.613): converges the fastest (before epoch 30), yet it reaches the *worst* test MAE. Fast training convergence does not imply good generalisation.
- **ReLU** (MAE 0.607, best): rapid, stable convergence with mild oscillations attributable to occasional dead-neuron events.
- **LeakyReLU** (MAE 0.608): virtually identical to ReLU. After `StandardScaler` normalisation the input distribution is centred, so the dying-neuron advantage of LeakyReLU is negligible.

Decision: ReLU for all subsequent experiments.

4.2.2 Weight Initialisation

Five initialisation schemes are compared, all paired with ReLU.

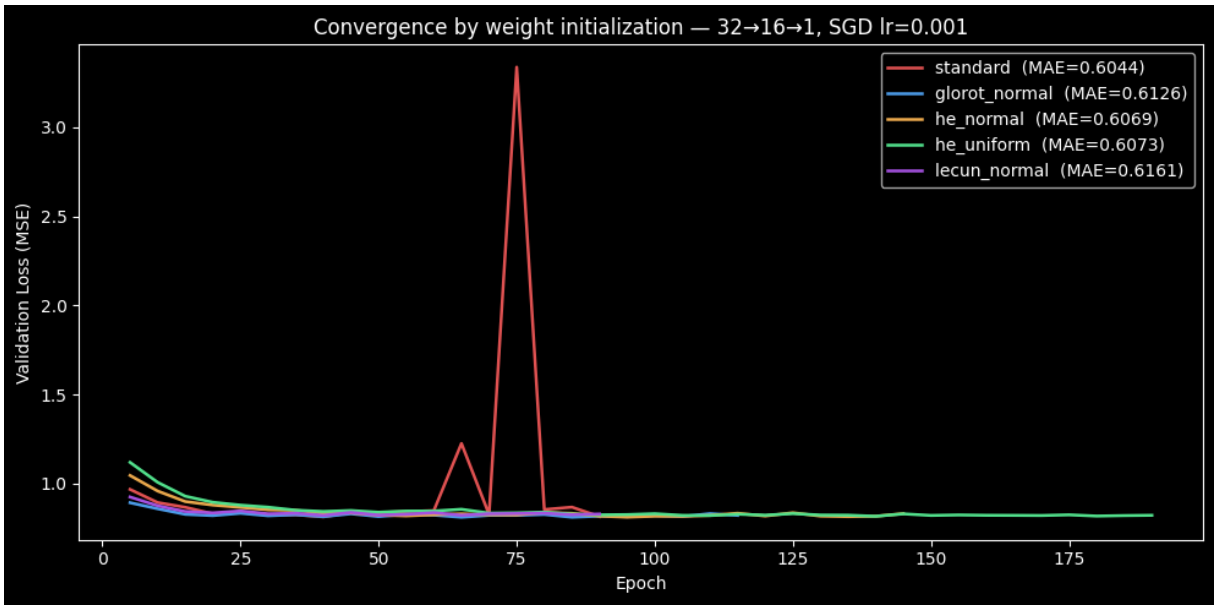

Figure 7: Convergence by initialisation scheme. Standard $\mathcal{N}(0,1)$ (red) triggers a gradient explosion at epoch ≈ 75 (MSE peaks at 3.3) but ultimately converges to the lowest test MAE. He normal/uniform converge stably and reach comparable performance.

Table 11: Final test MAE by initialisation scheme (ReLU, SGD $\alpha = 10^{-3}$).

Initialisation	MAE
Standard $\mathcal{N}(0,1)$	0.604
He normal	0.607
He uniform	0.607
Glorot normal	0.613
LeCun normal	0.616

Standard initialisation achieves the *lowest* final MAE but only through an unreliable mechanism: the gradient explosion near epoch 75 (MSE $\rightarrow$ 3.3) accidentally perturbs the weights out of a shallow basin. On a harder problem or a deeper network this spike would cause permanent divergence. He normal reaches near-identical performance (+0.003) with *monotone, predictable* convergence, and is theoretically motivated: for ReLU layers, $\mathrm{Var}(W) = 2/n_{\text{in}}$ (where n_{in} is the number of inputs to the layer) ensures that activation variance is preserved layer-to-layer, preventing both vanishing and exploding signals.
Decision: He normal for all subsequent experiments.

4.2.3 Optimizer and Momentum

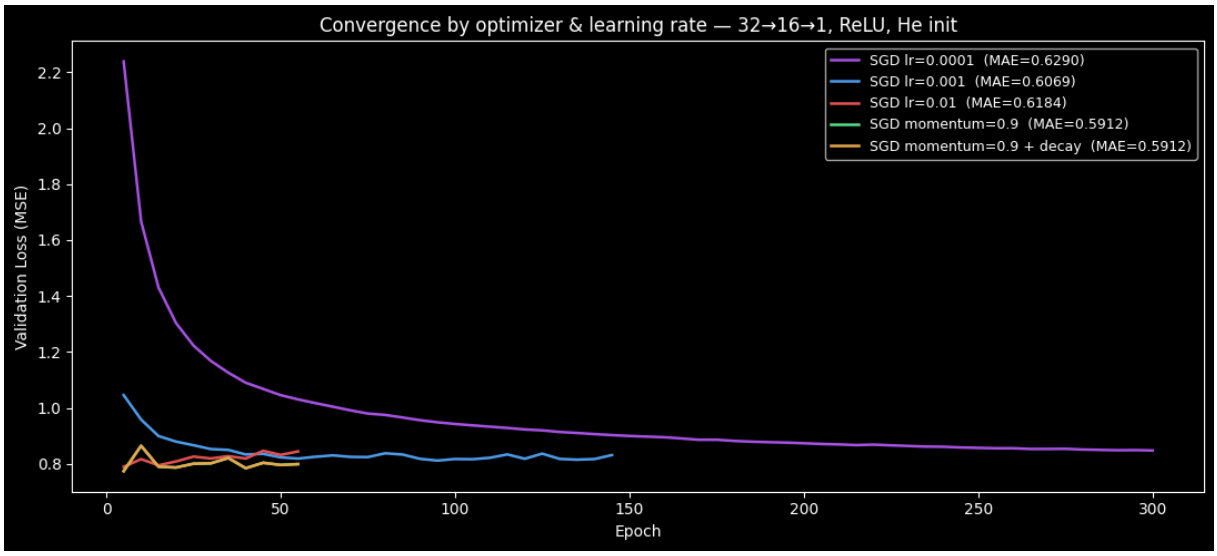

Figure 8: Convergence by optimizer. Plain SGD at three learning rates and SGD with momentum $\mu = 0.9$ (with and without LR decay). Momentum dominates all plain-SGD variants; adding decay changes nothing under early stopping.

Learning rate and effect of momentum.

- $\alpha = 10^{-4}$: convergence is too slow (MAE 0.629 at epoch 300, still declining).
- $\alpha = 10^{-3}$: stable convergence (MAE 0.607).
- $\alpha = 10^{-2}$: early overshoot followed by convergence to a worse minimum (MAE 0.618).
- SGD + momentum $\mu = 0.9$ (MAE 0.591): reaches the final MAE of plain SGD ($\alpha = 10^{-3}$) roughly $10\times$ faster, then converges further.
- SGD + momentum + LR decay (MAE 0.591): identical to plain momentum. Early stopping triggers before the decay schedule has a meaningful effect, rendering it redundant.

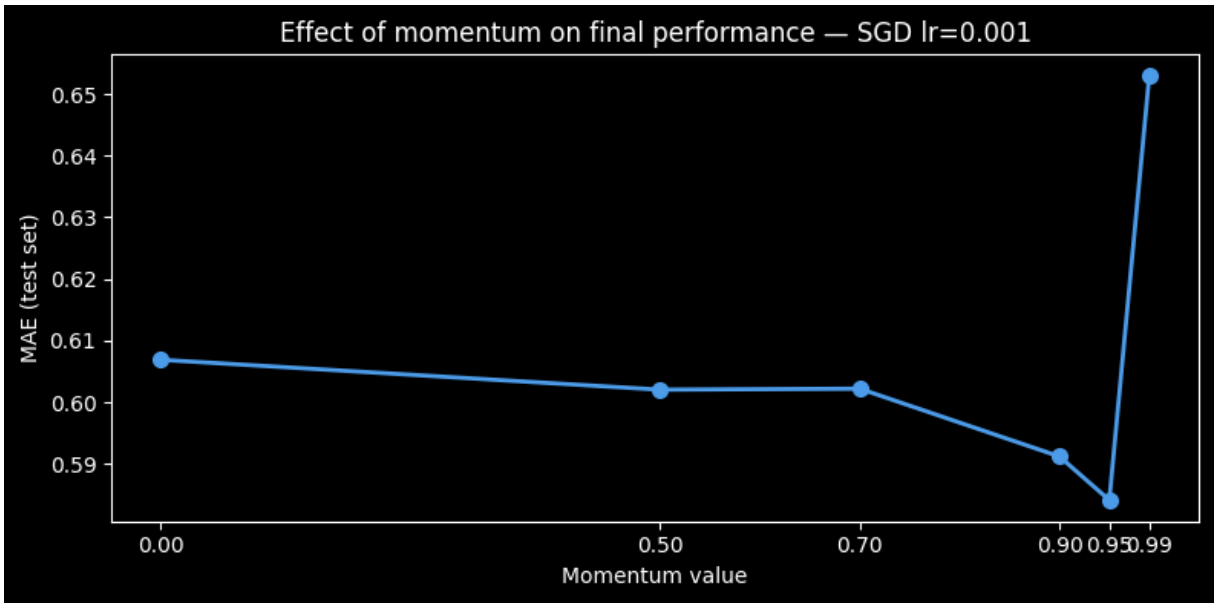

Figure 9: Test MAE as a function of momentum coefficient μ. Performance improves steadily to $\mu = 0.95$; at $\mu = 0.99$ the model diverges.

Table 12: MAE by momentum coefficient (He normal, ReLU, lag = 10).

μ	MAE
0.00	0.607
0.50	0.602
0.70	0.602
0.90	0.591
0.95	**0.587**
0.99	diverges

Momentum sweep. Performance improves steadily up to $\mu = 0.95$, then diverges at $\mu = 0.99$. At very high momentum the accumulated velocity overpowers the corrective gradient signal, causing the weights to oscillate with growing amplitude.
Decision: $\mu = 0.95$ for all subsequent experiments.

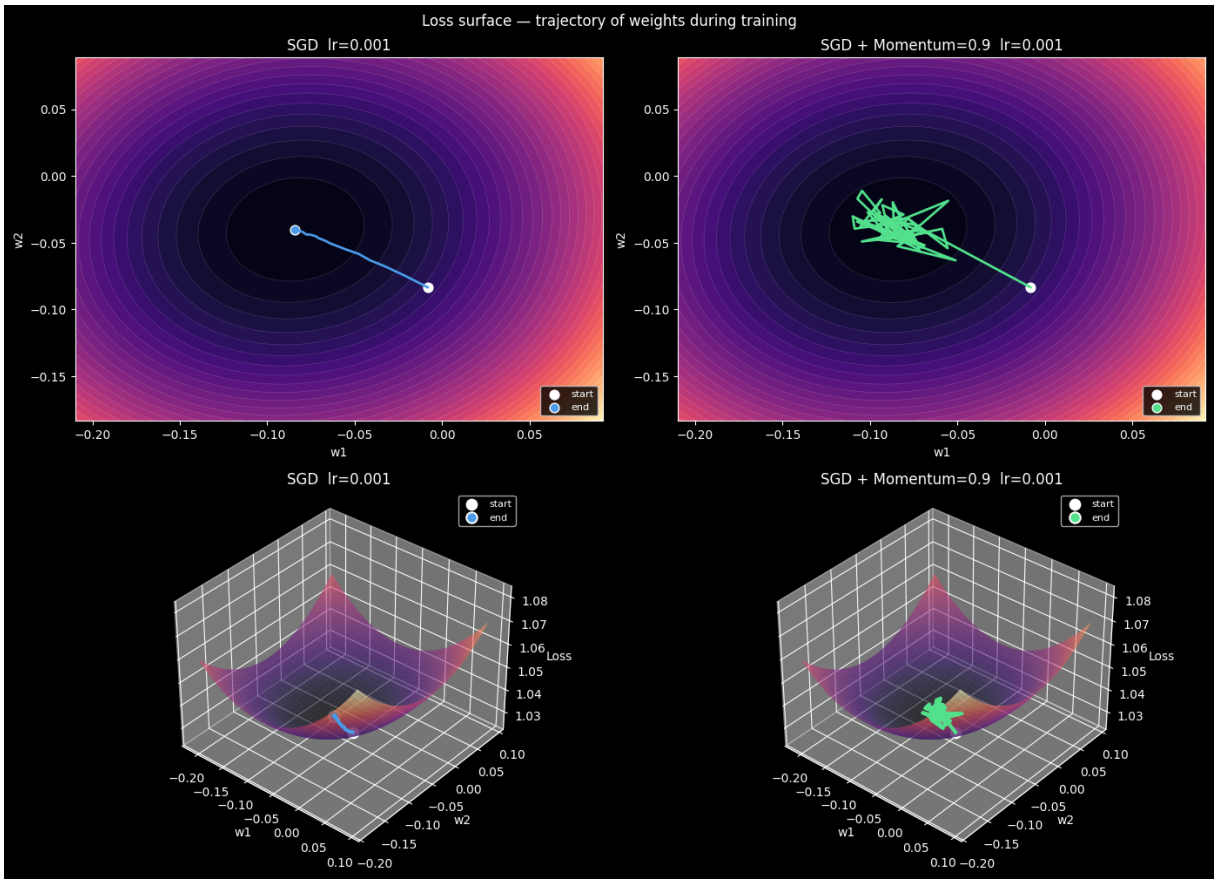

Figure 10: 2-D loss-surface contours (top) and 3-D surfaces (bottom) with weight trajectories for plain SGD (left, blue) and SGD+momentum $\mu = 0.9$ (right, green). Momentum follows a wider arc but descends to a darker (lower-loss) region.

Loss surface geometry. Plain SGD takes short, monotone gradient steps and stalls in the first basin it enters. Momentum accumulates velocity in the consistent gradient direction, allowing it to traverse flat saddle regions and settle in a deeper minimum — visible in the contour plot as the green trajectory reaching a darker zone unreachable by the blue one.

4.2.4 Temporal Window

The lag L is swept across $\{1, 3, 5, 10, 20, 30, 45, 60\}$ using the best configuration found above (ReLU, He normal, SGD $\mu = 0.95$).

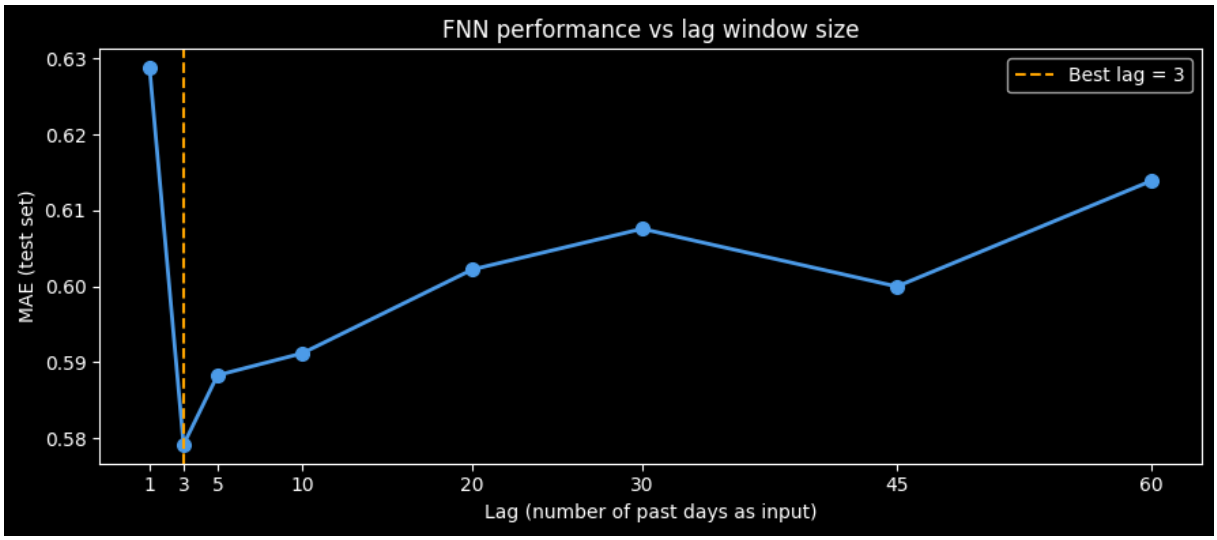

Figure 11: Test MAE as a function of lag window L. Performance peaks at $L = 3$ (MAE 0.578, dashed orange); it generally increases for larger windows with a local dip at lag 45.

Table 13: Test MAE by lag window (ReLU, He normal, SGD $\mu = 0.95$).

Lag L	MAE
1	0.629
3	**0.578**
5	0.589
10	0.591
20	0.602
30	0.608
45	0.600
60	0.614

Performance peaks at $L = 3$ and generally degrades beyond, though not strictly monotonically (lag = 45 recovers slightly below lag = 30, a seed-dependent fluctuation). The pattern reflects a structural limitation of the FNN: all lag features receive independently learned fixed weights, so the network has no mechanism to discount distant, noisy lags. Beyond 3 days, the additional information added by older returns is outweighed by the noise they introduce.
Decision: $L = 3$ for all subsequent experiments.

4.3 Diagnostics: Inside the Trained Network

The figures in this section do not drive hyperparameter choices; they explain *why* the chosen configuration (ReLU + He normal) works and why the bad configuration (Sigmoid + standard init) fails.

4.3.1 Weight and Activation Distributions

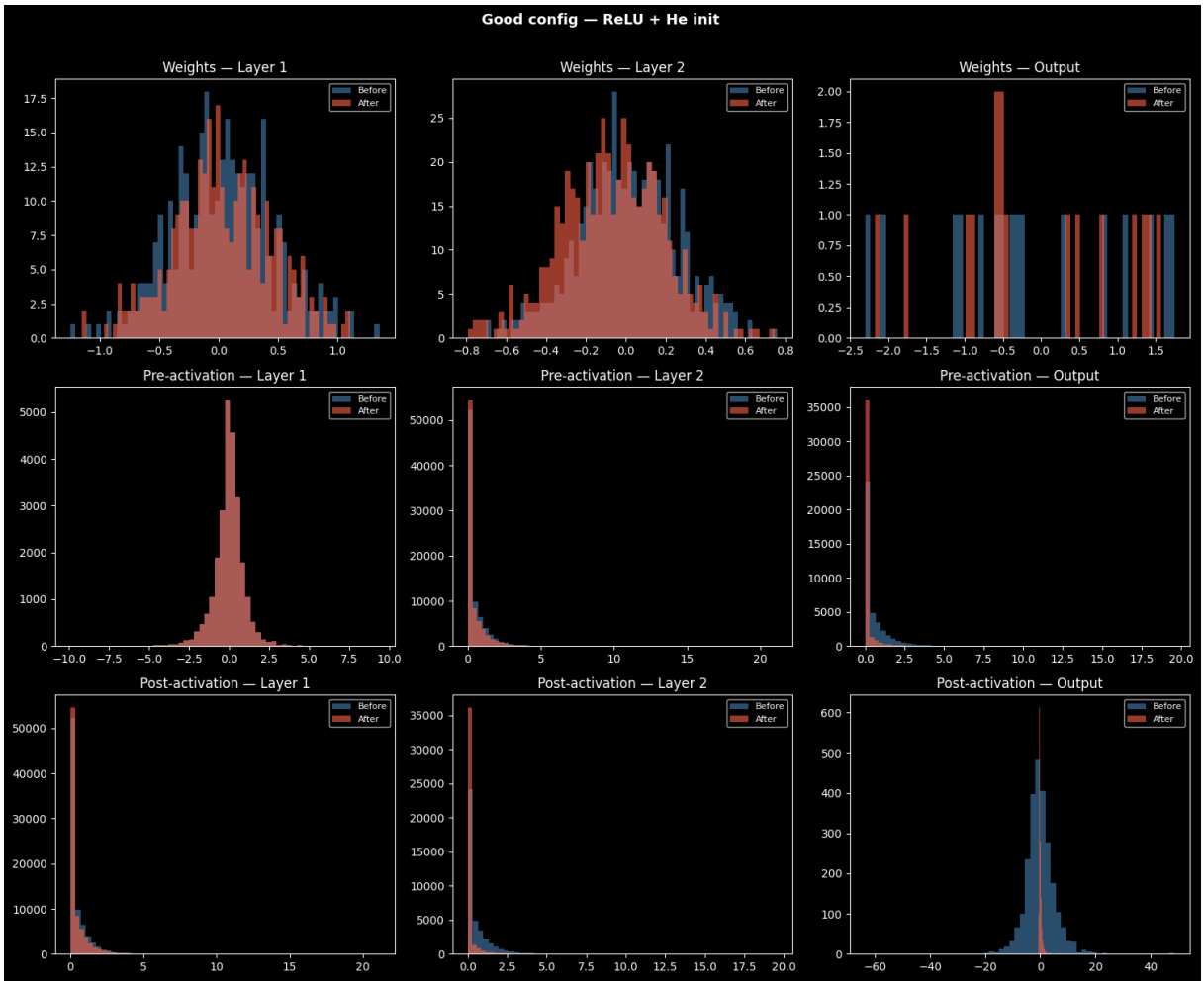

Figure 12: *Good* configuration (ReLU + He normal). Rows: weight distributions, pre-activations, post-activations. Columns: layers 1, 2, output. Blue = before training; red = after. All three layers show substantial weight spreading and healthy activation ranges.

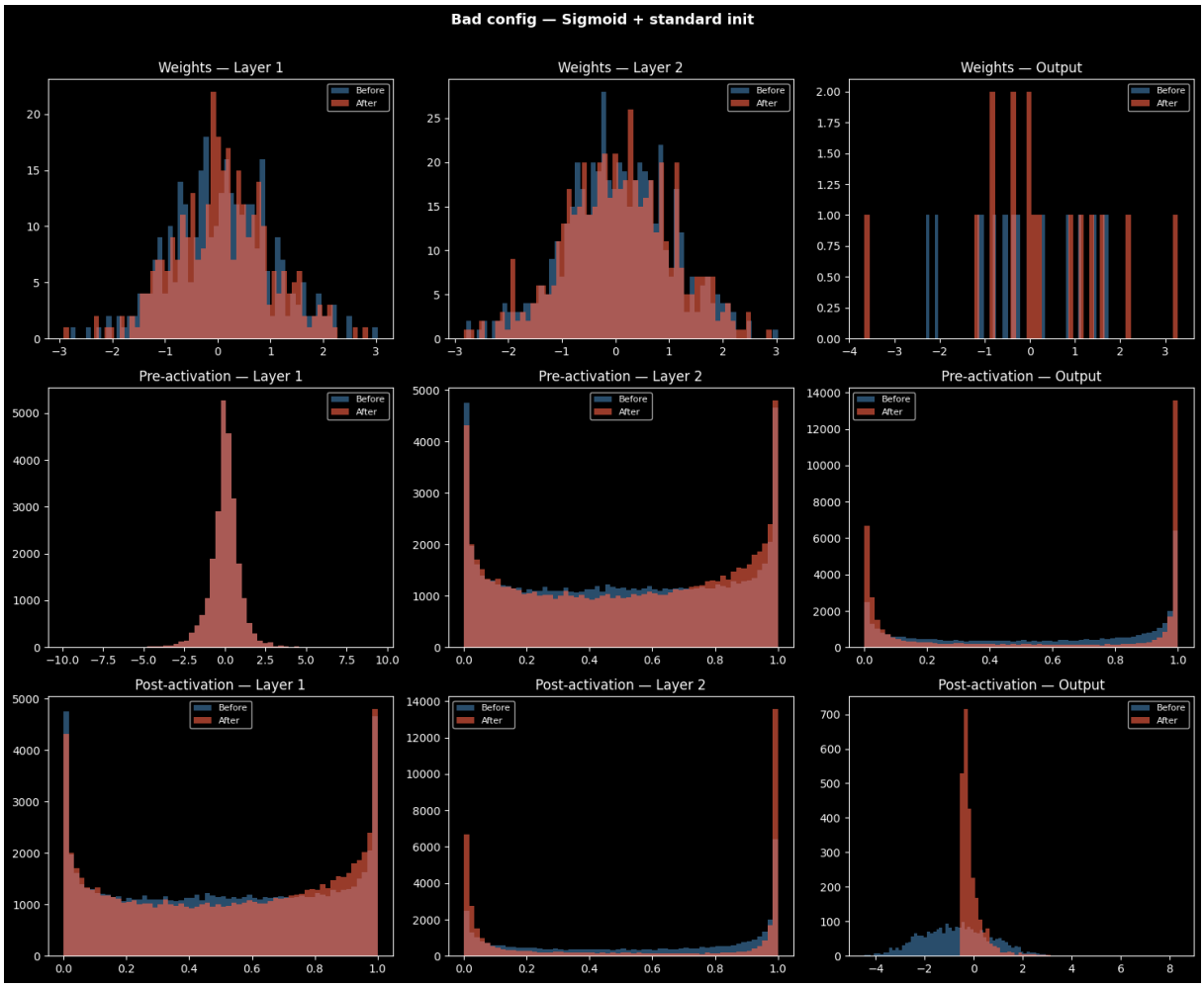

Figure 13: *Bad* configuration (Sigmoid + standard init). Same layout. Weights barely move during training; layer 2 post-activations collapse to the narrow interval [0.9, 1.0] — the saturation zone — transmitting essentially no information to the output.

With ReLU + He, weight distributions broaden during training and pre-activations span a wide range: every neuron is active and contributing. With Sigmoid + standard init, the gradient explosion compresses post-activations into Sigmoid's saturation zone; backpropagated gradients become near-zero and learning stalls. The distribution collapse renders vanishing gradients directly visible without gradient inspection.

4.3.2 Gradient Flow

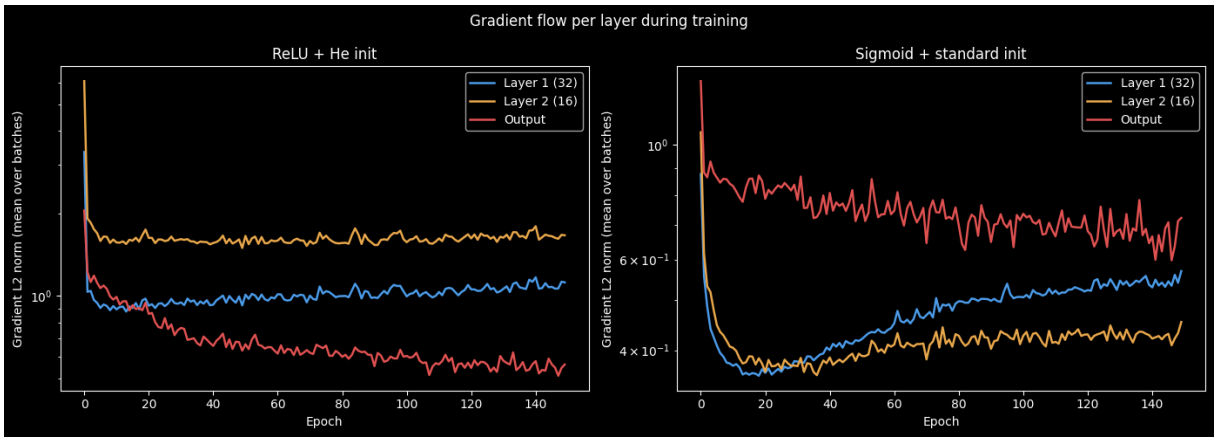

Figure 14: Gradient ℓ_2-norms per layer over 150 epochs. Left (ReLU + He): all layers maintain comparable norms throughout. Right (Sigmoid + standard): early layers receive gradients 2–3× smaller than the output layer.

With ReLU, the derivative equals 1 for every active neuron, so gradients propagate backward without attenuation: all layers update at the same scale. With Sigmoid, the derivative is bounded by 0.25 and approaches 0 in saturation; compounded across layers this concentrates learning at the output while the input layer stagnates. In a deeper network this effect would be catastrophic.

4.3.3 Weight Evolution During Training

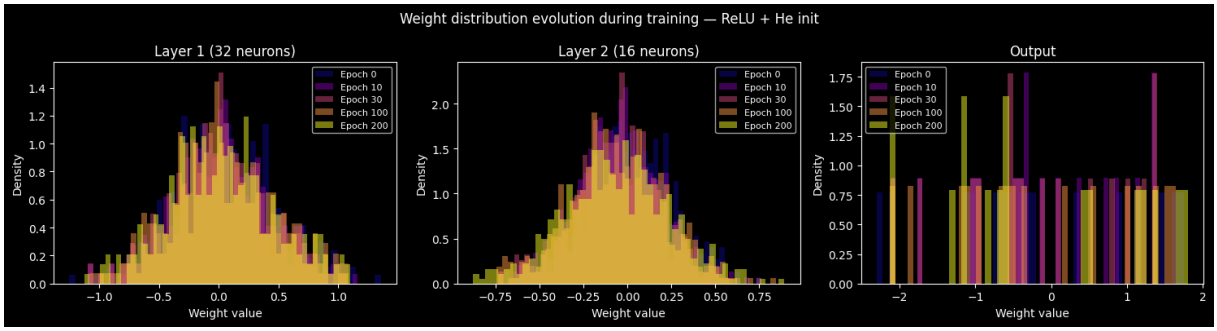

Figure 15: Weight distribution snapshots at epochs 0, 10, 30, 100, 200 (ReLU + He normal, best config). All three layers exhibit progressive spreading from a tight initialisation to broad, learned distributions.

At epoch 0 the He initialisation produces narrow Gaussian-like distributions in all layers. By epoch 200, layers 1 and 2 have spread considerably and the output layer has developed a multimodal structure. The monotone spreading across every layer confirms that no part of the network is frozen or dead: gradient updates reach and reshape all weights throughout training.

4.4 Structural Limitations

Despite a 9 % improvement over the linear baseline, the FNN has two architectural limitations that no amount of hyperparameter tuning can address.

4.4.1 Order Blindness

The FNN receives the L lag features as a flat, unordered vector. To measure how much temporal ordering it *implicitly* exploits, we evaluate the trained model on 30 random permutations of the input features and report the mean MAE gap.

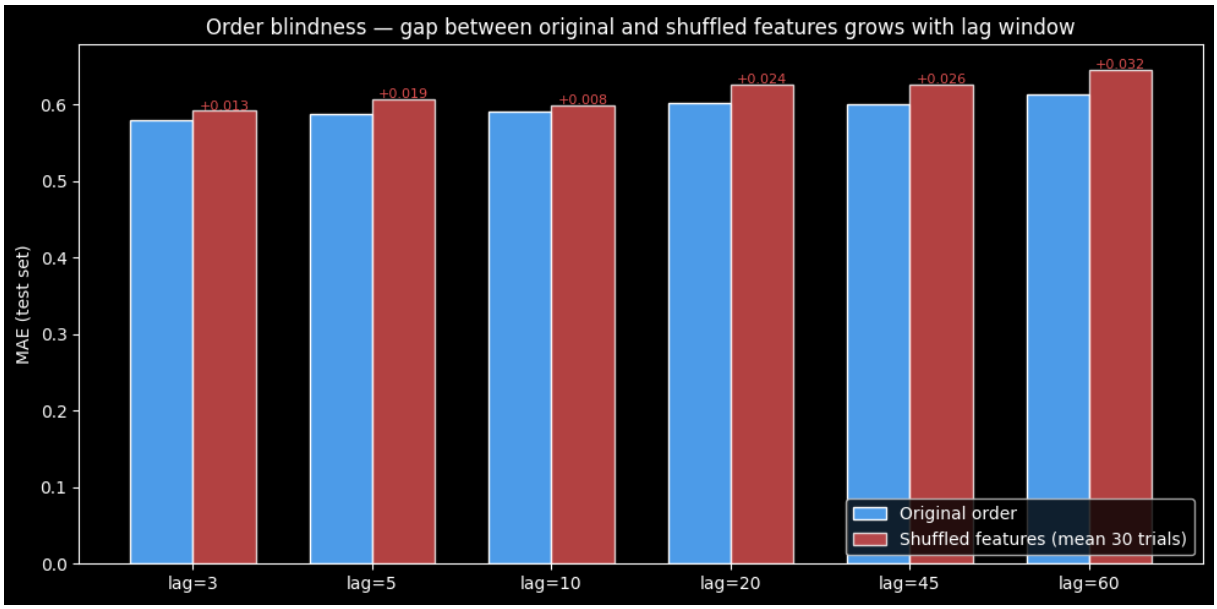

Figure 16: Test MAE for original feature order (blue) vs. randomly shuffled features (red, mean of 30 trials) at each lag window. The degradation gap grows from +0.013 at lag 3 to +0.032 at lag 60.

Table 14: Original vs. shuffled-input MAE by lag window.

Lag L	Original	Shuffled	Gap
3	0.578	0.591	+0.013
5	0.589	0.608	+0.019
10	0.591	0.599	+0.008
20	0.602	0.626	+0.024
45	0.600	0.626	+0.026
60	0.614	0.646	+0.032

The network has learned to exploit recency through its weight magnitudes — recent lags receive larger learned weights — but this is an emergent property of optimisation, not a guarantee of the architecture. The growing gap with lag window confirms that richer temporal structure is present and the FNN partially uses it; a recurrent architecture would process inputs sequentially, making temporal ordering a structural rather than a learned property.

4.4.2 Regime Blindness

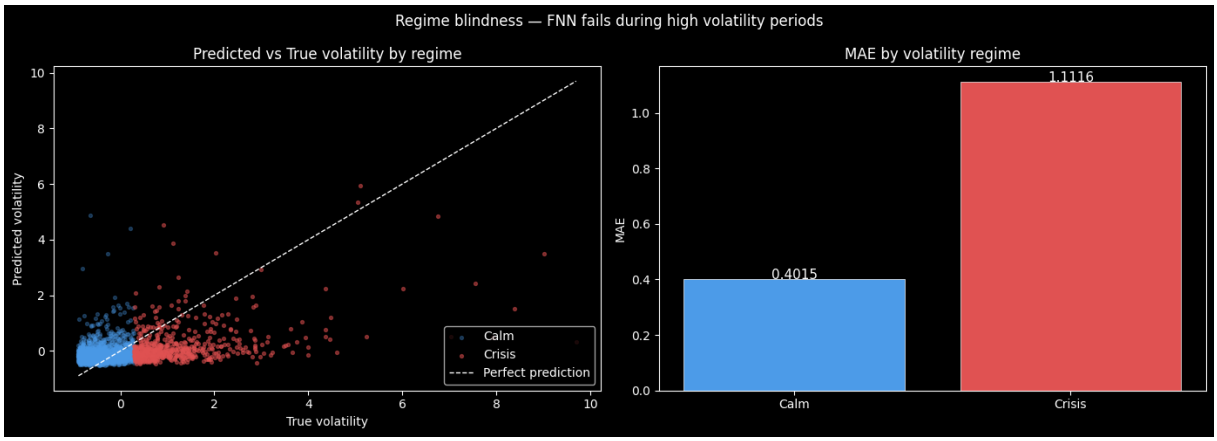

Figure 17: Left: predicted vs. true volatility coloured by regime (calm = blue, crisis = red). Crisis predictions cluster near zero regardless of the true spike magnitude. Right: MAE by regime — crisis MAE is 2.77× higher than calm MAE.

Table 15: FNN MAE by volatility regime (75th-percentile threshold, $L = 3$; note: the Data section uses a 90th-percentile definition).

Regime	MAE	Share
Calm ($\sigma_t < \sigma^*_{75}$)	0.402	75 %
Crisis ($\sigma_t \geq \sigma^*_{75}$)	1.112	25 %
Crisis / Calm ratio	2.77×	

The FNN makes each prediction from a static 3-day snapshot and then resets its state entirely. It cannot accumulate the context of a sustained volatility cluster building over days or weeks. Without persistent memory, *"high volatility today ⇒ high volatility tomorrow"* is not learnable.

4.5 Summary

Table 16: Best FNN configuration and rationale.

Parameter	Choice	Rationale
Activation	ReLU	Best test MAE; stable gradient propagation
Initialisation	He normal	Stable convergence; variance-preserving for ReLU
Optimizer	SGD $\mu = 0.95$	Fastest convergence; best generalisation
Lag window	$L = 3$	Beyond 3 days, distant lags add noise

With this configuration the FNN achieves **MAE** = 0.579 on the test set versus **0.638** for the linear baseline — a **9 % improvement** driven entirely by the non-linear activation resolving symmetry blindness.

Two structural limitations, demonstrated empirically, motivate the architectures developed in the following sections:

- **Order blindness**: the FNN has no positional encoding; a recurrent architecture is needed to make temporal ordering a structural property rather than an optimisation artefact.
- **Regime blindness**: without a persistent hidden state, the model cannot exploit volatility clustering; an RNN is needed to carry context across time steps.

5 Recurrent Neural Network

5.1 From FNN to RNN

The FNN section identified two structural limitations: *order blindness* (the lag vector has no positional encoding, so temporal ordering is only partially exploited through learned weight magnitudes) and *regime blindness* (the model resets entirely between predictions and cannot accumulate context across days). A Recurrent Neural Network (RNN) addresses both by construction.

Three properties of a flat-window FNN make it unsuitable as sequence length grows:

- **Order blindness.** Each lag receives an independently learned fixed weight; shuffling the input features degrades performance but does not break it — temporal ordering is an optimisation artefact, not a structural guarantee.
- **Fixed hard cutoff.** The window size must be chosen upfront; any dependency beyond it is invisible to the model.
- **Parameter explosion.** Every additional lag adds a full row of input weights, making long contexts expensive.

An RNN processes the sequence one time step at a time, maintaining a *hidden state* $\mathbf{h}_t \in \mathbb{R}^H$ (where H is the hidden size) that acts as a compressed memory of all previous inputs:

$$\mathbf{h}_t = \tanh\big(W_{ih}\,\mathbf{x}_t + W_{hh}\,\mathbf{h}_{t-1} + \mathbf{b}\big), \qquad \mathbf{h}_0 = \mathbf{0},$$

where $W_{ih} \in \mathbb{R}^{H\times d}$ maps the current input, $W_{hh} \in \mathbb{R}^{H\times H}$ is the *recurrence matrix* that propagates the hidden state, and d is the input dimension. Temporal ordering is now structural: the same W_{ih} and W_{hh} are applied at every step, so the position of $\mathbf{x}_t$ in the sequence determines which historical context $\mathbf{h}_{t-1}$ it is combined with.

Vanishing gradient problem. During backpropagation through time (BPTT), the gradient with respect to a hidden state $T-t$ steps in the past is multiplied by $(W_{hh}^\top)^{T-t}$. If the spectral radius $\rho(W_{hh}) < 1$, this product shrinks exponentially and long-range gradients vanish; if $\rho(W_{hh}) > 1$, they explode. In practice, tanh saturation further attenuates gradients, limiting effective credit assignment to roughly 10–20 steps. Gradient clipping is the standard palliative: it caps the gradient norm before each update, preventing explosion without curing vanishing.

5.2 Architecture and Training Setup

Architecture

The model feeds a sequence of $T = 20$ scalar inputs (one-day log-returns) into a single-layer RNN with hidden size $H = 64$ and tanh nonlinearity, then maps the final hidden state $\mathbf{h}_T$ to a scalar volatility forecast through a linear head:

$$\big(x_1, \ldots, x_{20}\big) \xrightarrow{\text{RNN}_{64,\tanh}} \mathbf{h}_{20} \in \mathbb{R}^{64} \xrightarrow{\text{Linear}} \hat{y} \in \mathbb{R}_+.$$

The head uses Xavier uniform initialisation and a zero bias; no dropout is applied to the baseline.

Training

Table 17: RNN baseline training configuration.

Hyperparameter	Value
Loss	MSE
Optimizer	Adam
Learning rate	5×10^{-4}
Batch size	64
Max epochs	200
Early stopping	patience 30 (checked every epoch)
Gradient clipping	$\|\mathbf{g}\|_2 \leq 0.5$

Regime-Aware Test Set

A single aggregate MAE conceals where the model excels and where it breaks down. Three non-overlapping held-out periods are selected to represent distinct market regimes:

Table 18: Regime-aware test set composition.

Regime	Period	Calendar days	Trading days
Calm	2017 (full year)	251	231
Clustering	2022 (full year)	251	231
Spikes	Mar–Apr 2020 (COVID)	43	23

The calm regime (2017) consists of small, stable daily moves. The clustering regime (2022) features persistently elevated volatility driven by the Fed rate-hike cycle. The spikes regime (Mar–Apr 2020) captures the COVID crash: extreme isolated moves never seen in training, a deliberate out-of-distribution test.

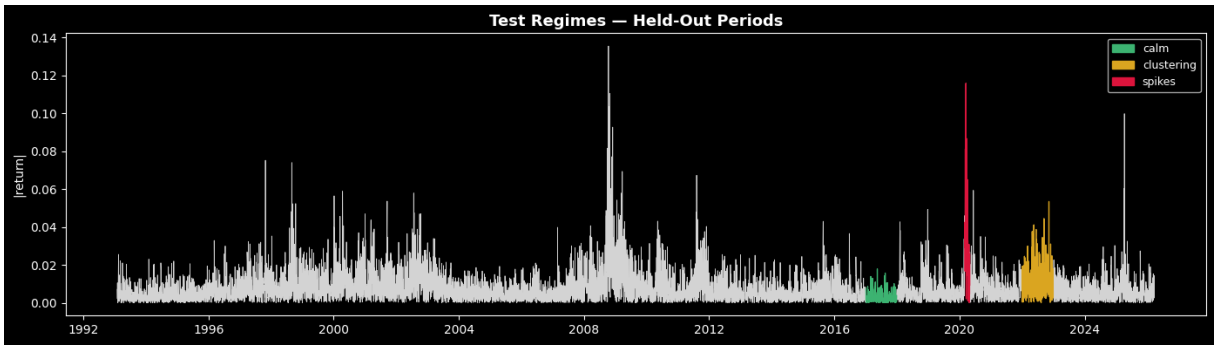

Figure 18: Three regime periods highlighted on the full S&P 500 log-return series. Calm (2017): low, stable moves. Clustering (2022): sustained elevated volatility. Spikes (Mar–Apr 2020): extreme COVID-crash moves.

5.3 Baseline Results

Training converges at epoch 58 (early stopping) with a best validation loss of 0.542.

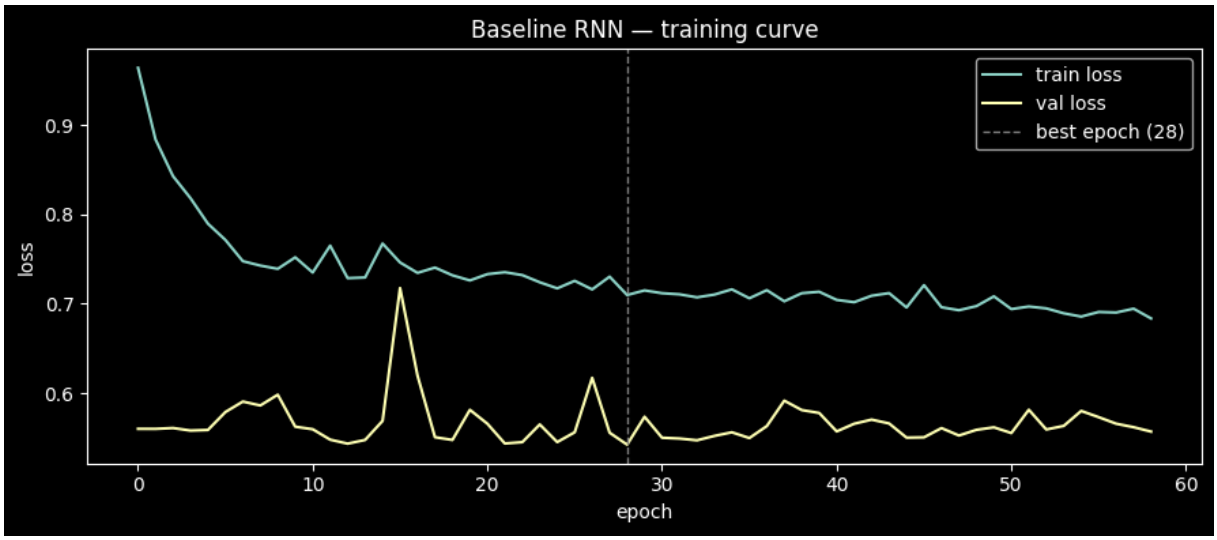

Figure 19: Training and validation loss curves. Early stopping triggers at epoch 58; the gap between train and validation loss indicates mild overfitting that early stopping contains.

The overall test MAE is **0.00434** (original log-return scale) or equivalently **0.502** in the standardised space used by the FNN section — a **13 %** improvement over the best FNN (MAE = 0.579, lag = 3).

Predicted vs True Distribution

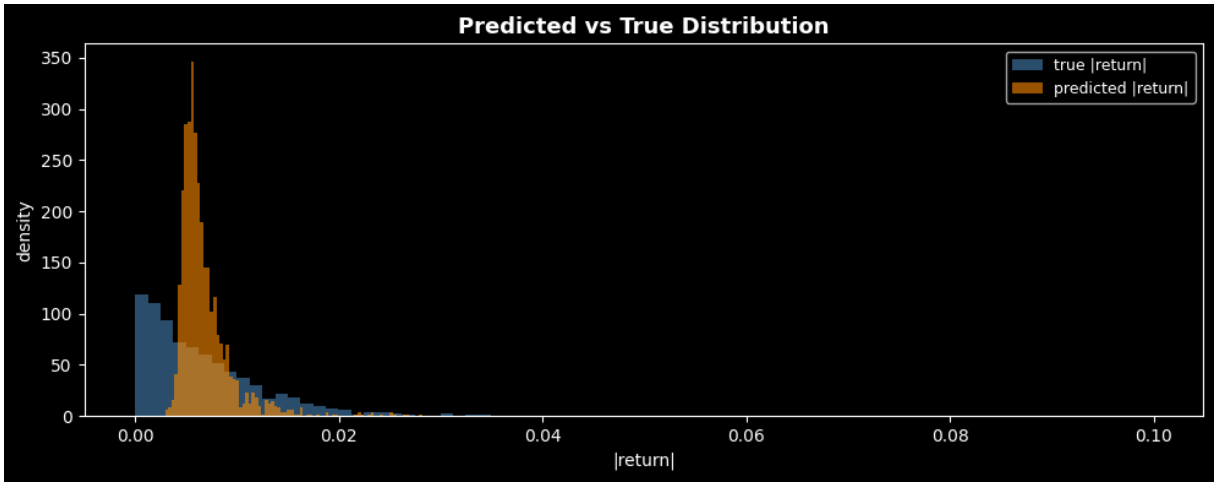

Figure 20: Predicted vs. true volatility distribution on the validation set. The predicted distribution is visibly narrower: the model compresses outputs toward the mean and systematically under-represents the tails.

The predicted distribution is narrower than the true one: the model compresses its outputs toward the mean and under-represents extreme volatility days. This *mean-reversion bias* is a structural consequence of MSE training, which penalises large deviations quadratically and therefore incentivises central-tendency predictions.

Regime Breakdown

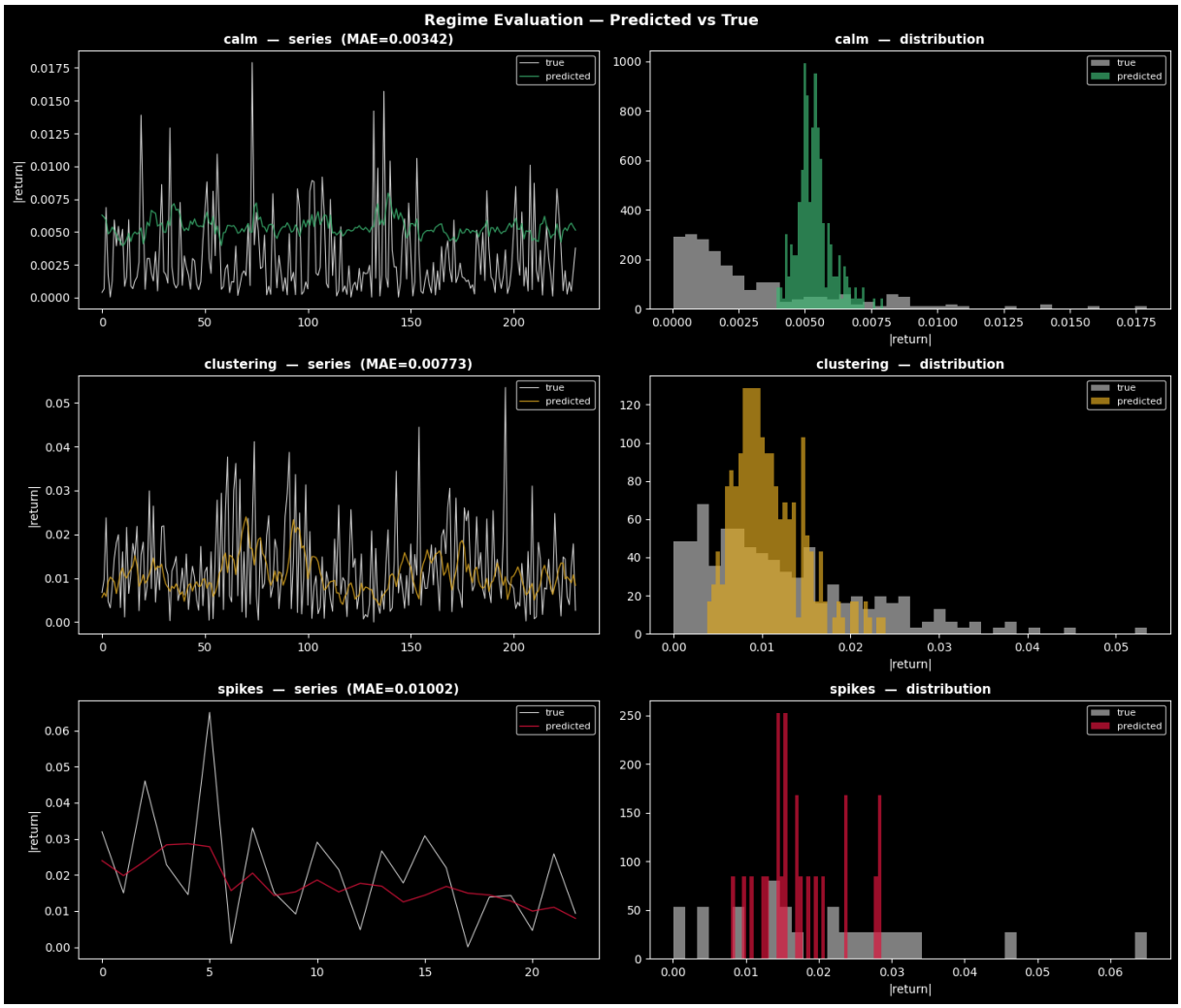

Figure 21: Per-regime evaluation. For each regime: predicted vs. true volatility time series (left) and distribution comparison (right). Errors grow from calm to clustering to spikes, in line with the mean-reversion bias.

Table 19: RNN MAE by market regime.

Regime	Trading days	MAE
Calm	231	0.003423
Clustering	231	0.007733
Spikes	23	0.010021
Spikes / Calm ratio		2.93×

The spikes regime is worst by a factor of 2.93× relative to calm — consistent with the mean-reversion bias: COVID-level moves were never seen during training, so the model has no basis for outputting values of that magnitude.

5.4 Diagnostics: What Did the Model Learn?

Three complementary angles reveal whether the recurrent pathway is genuinely contributing: weight distributions, hidden state activation levels, and input sensitivity across sequence positions.

5.4.1 Weight Distributions

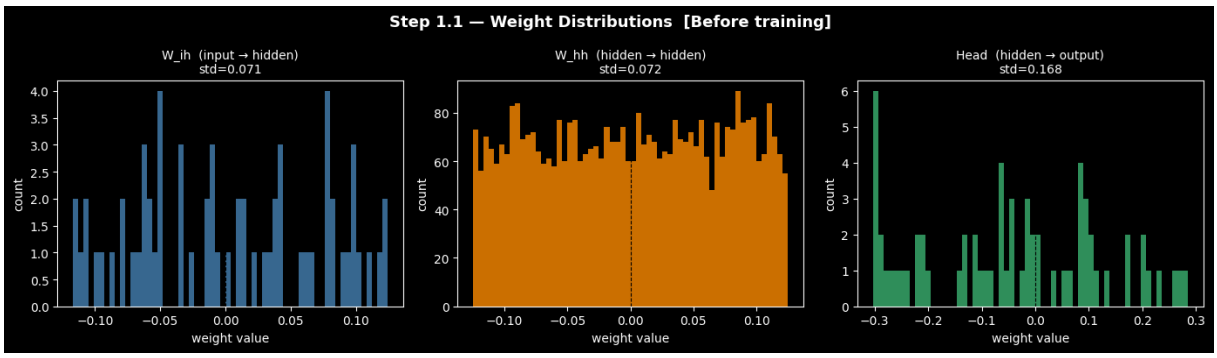

Figure 22: Weight distributions before training. W_{ih} (input→hidden), W_{hh} (hidden→hidden, the memory matrix), and the linear head. All initialised in tight distributions around zero.

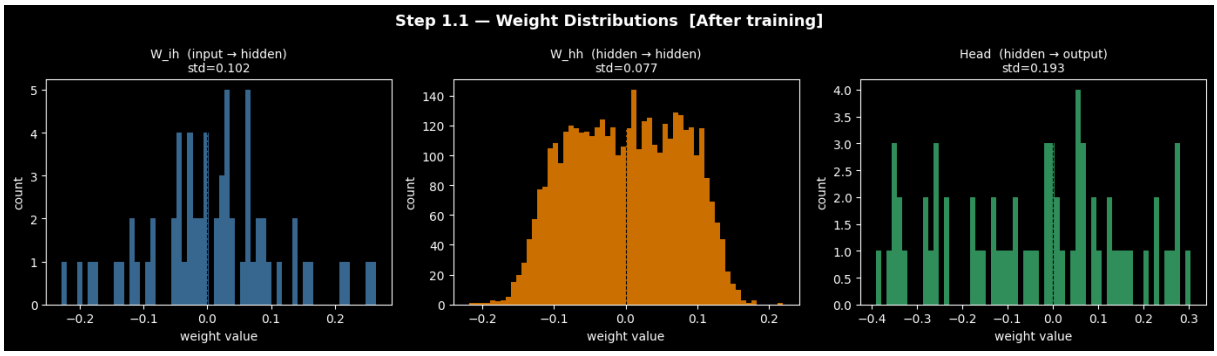

Figure 23: Weight distributions after training. W_{hh} shows the most pronounced reshaping: the distribution widens and develops structure, confirming that the recurrent pathway is actively used by the trained model.

W_{hh} is the most visibly reshaped matrix after training: its distribution widens and develops internal structure, while W_{ih} and the head weights shift only modestly. This confirms that the recurrent pathway — not merely the input projection — is genuinely contributing to the model's predictions.

5.4.2 Hidden State Activity

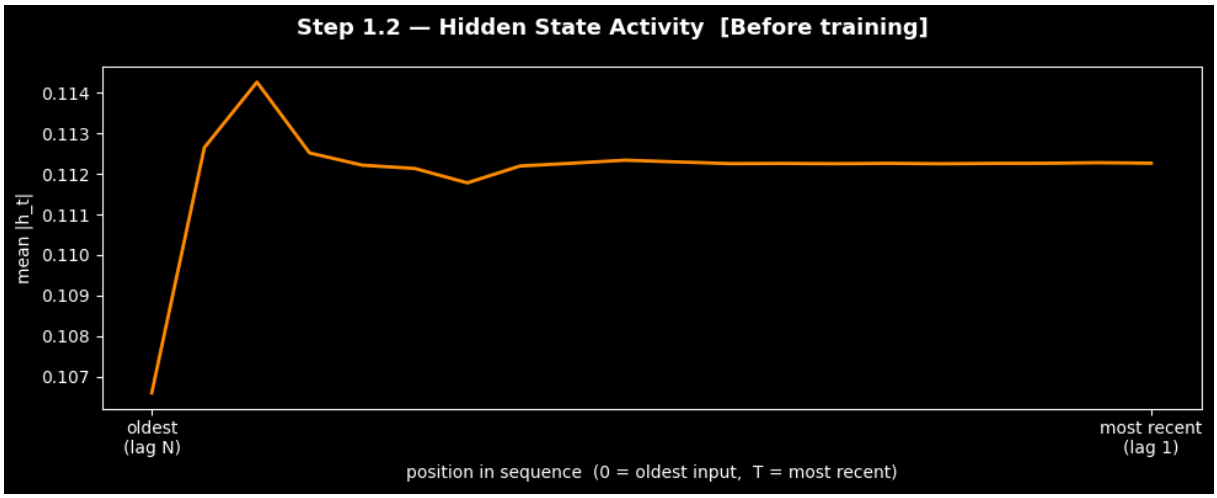

Figure 24: Average hidden state activation at each sequence position before training. Activity rises from the zero initialisation then plateaus, reflecting the warmup from $\mathbf{h}_0 = \mathbf{0}$.

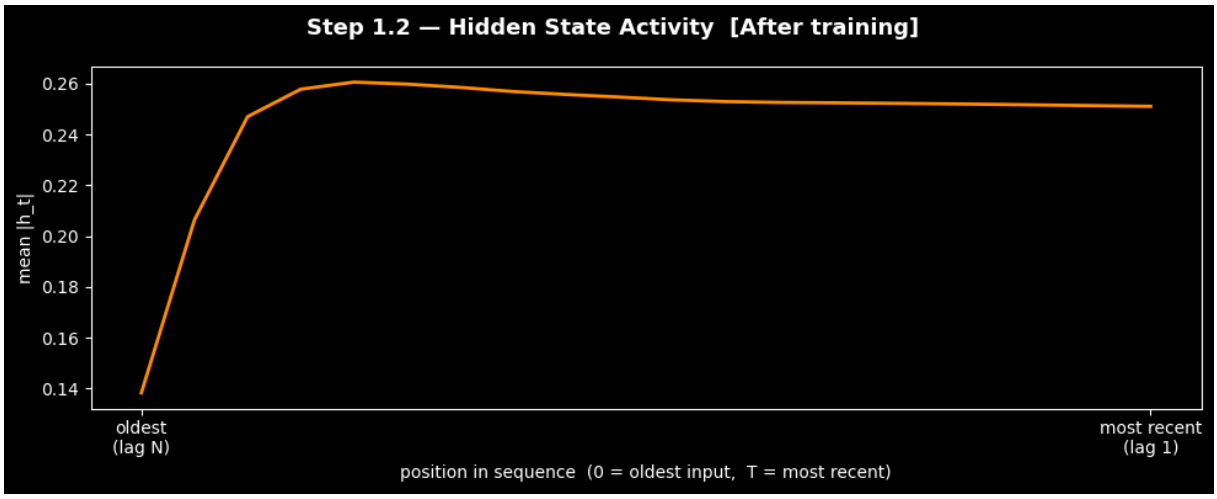

Figure 25: Average hidden state activation after training. The shape (rise then plateau) is unchanged, but the saturation level has increased from ≈ 0.125 to ≈ 0.325 — a $2.5\times$ amplification — indicating that training taught the network to store more information in each hidden state.

The shape of the activation profile is identical before and after training (rise from zero then plateau), but the scale changes dramatically: saturation level increases from ≈ 0.125 to ≈ 0.325 ($2.5\times$). The plateau is reached after roughly 20–30 % of the sequence, suggesting that effective memory is concentrated in the most recent steps despite the full 20-lag window.

5.4.3 Input Sensitivity by Lag

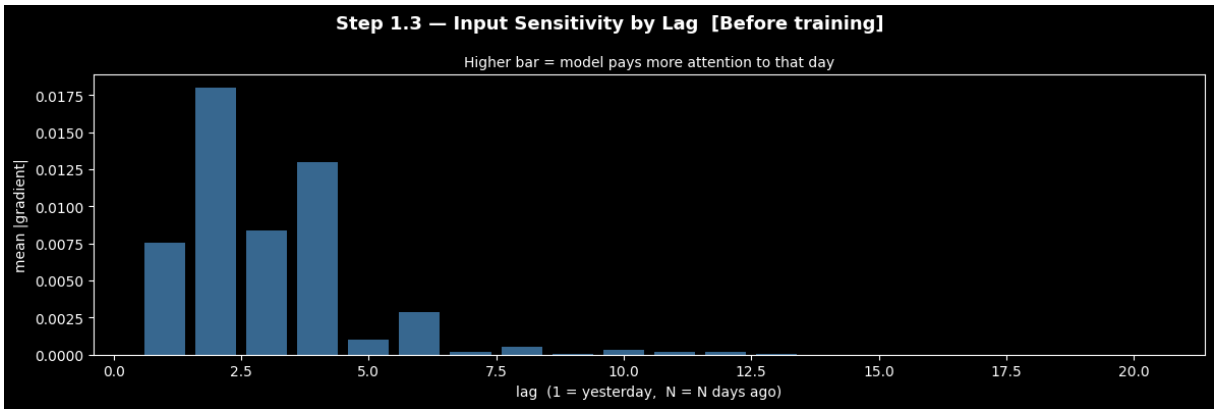

Figure 26: Input sensitivity before training: how much does perturbing $\mathbf{x}_t$ move the final prediction? Sensitivity collapses at lags 2–3; the model is effectively ignoring all but the most recent inputs.

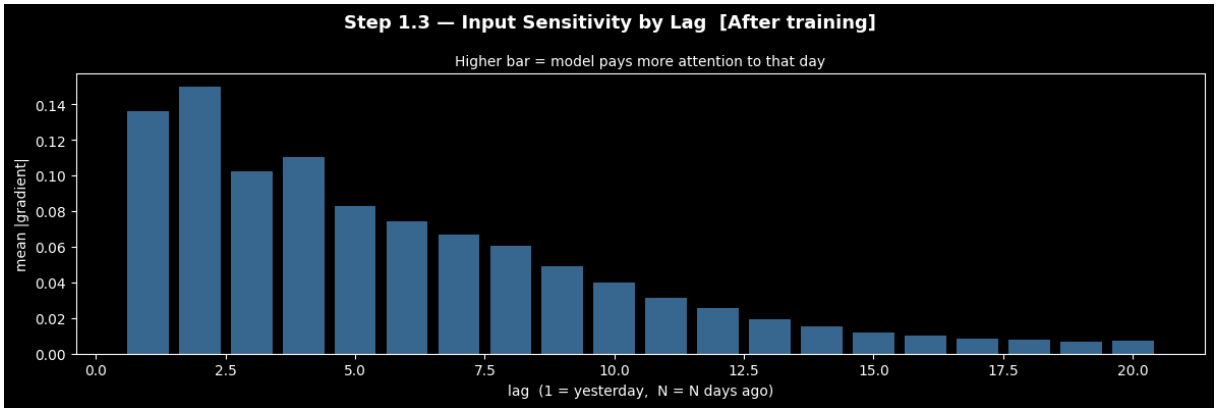

Figure 27: Input sensitivity after training. Sensitivity is now distributed across the full 20-lag window, demonstrating that training has taught the recurrent pathway to route historical context to the final prediction.

Before training, sensitivity collapses at lags 2–3: the untrained model is effectively ignoring all but the most recent input. After training, sensitivity is spread across the full 20-lag window. This is the clearest evidence that the recurrent pathway is working: training redistributed gradient-driven credit across the entire sequence.

5.5 Failure Analysis

5.5.1 Residuals Over Time

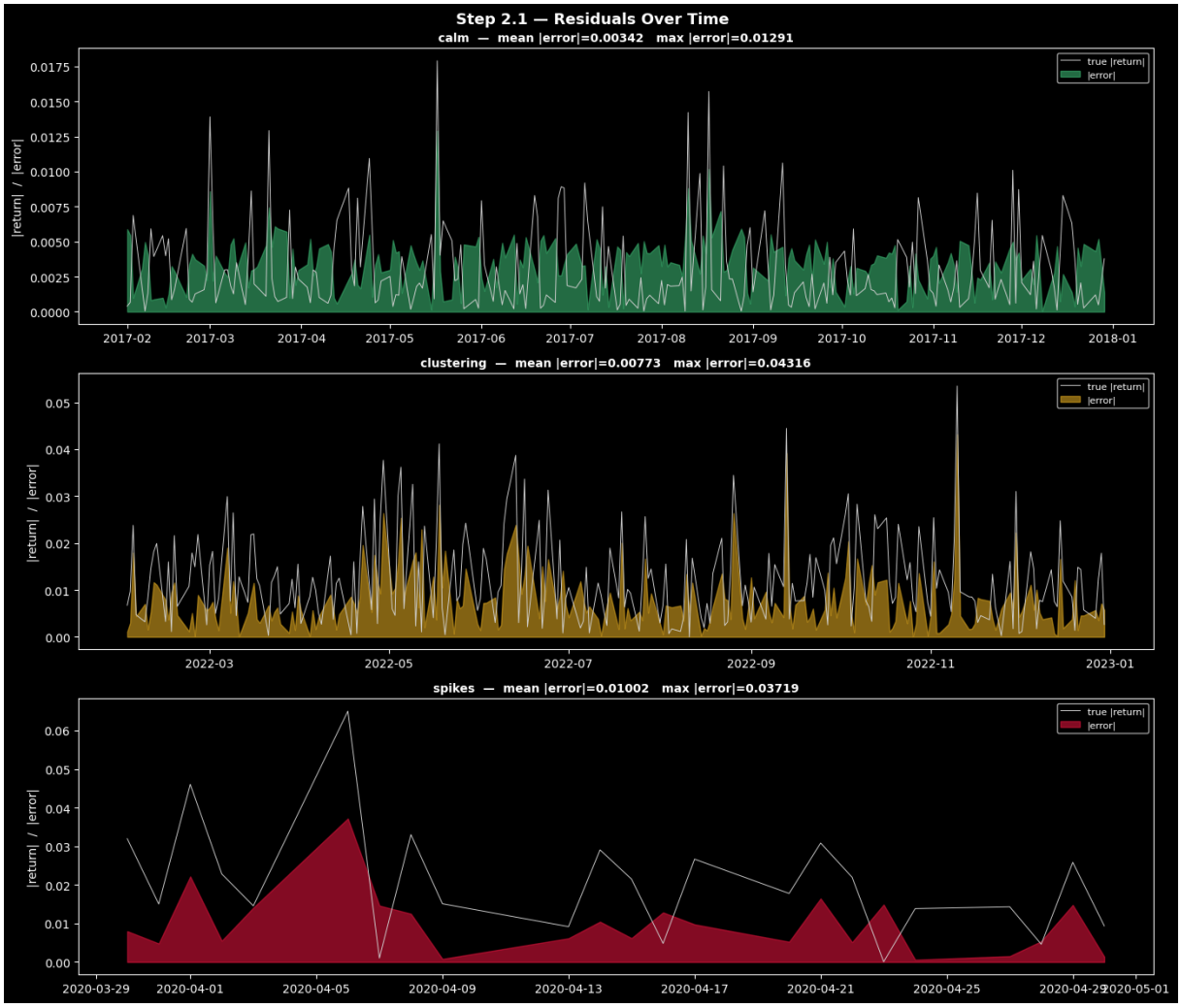

Figure 28: Residuals over time by regime. White line: true absolute return. Coloured area: absolute prediction error $|y - \hat{y}|$. The error envelope tracks the true volatility signal: small and stable in the calm regime, elevated in clustering, and spiking sharply during the COVID crash.

The error envelope directly tracks the magnitude of true volatility: residuals are small and stable in the calm regime, persistently elevated in clustering, and spike sharply during the COVID crash. Error variance is proportional to volatility magnitude — the model's worst moments coincide exactly with the market's worst moments.

5.5.2 Error vs Magnitude

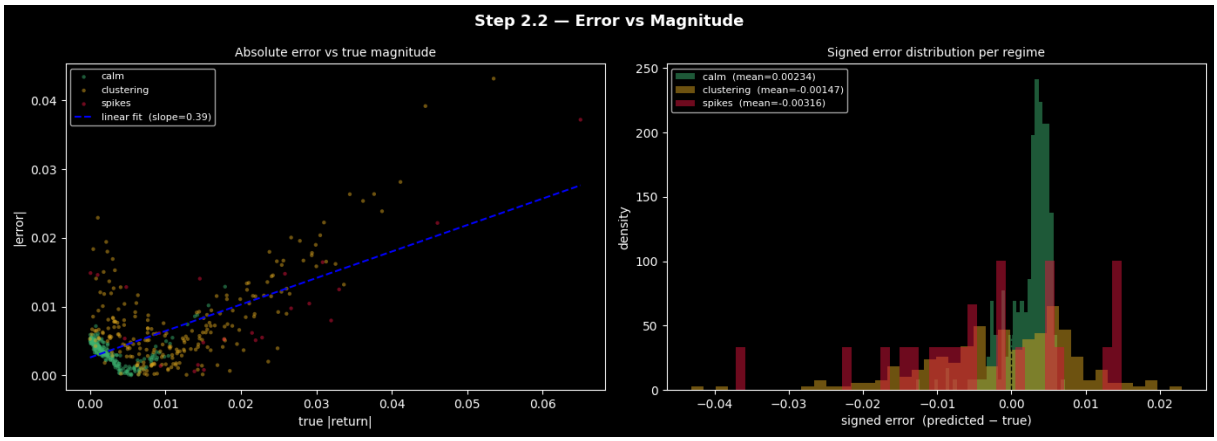

Figure 29: True volatility vs. prediction error, coloured by regime. The positive slope confirms mean-reversion bias across all regimes. Histograms (right) show the direction of the bias per regime: the spikes histogram is shifted furthest left (systematic under-prediction); the calm histogram is nearly centred.

The scatter plot reveals a positive slope between true volatility and prediction error across all three regimes: the larger the true move, the larger the error. The per-regime histograms confirm the direction: the spikes regime is shifted furthest toward negative errors (systematic under-prediction), while calm predictions are nearly unbiased. The model is most wrong on the days that matter most.

5.5.3 Worst Predictions Per Regime

Each panel shows the worst 5 % of predictions for a given regime: the 20-step input lookback (blue), the true target (purple), and the model prediction (red dot).

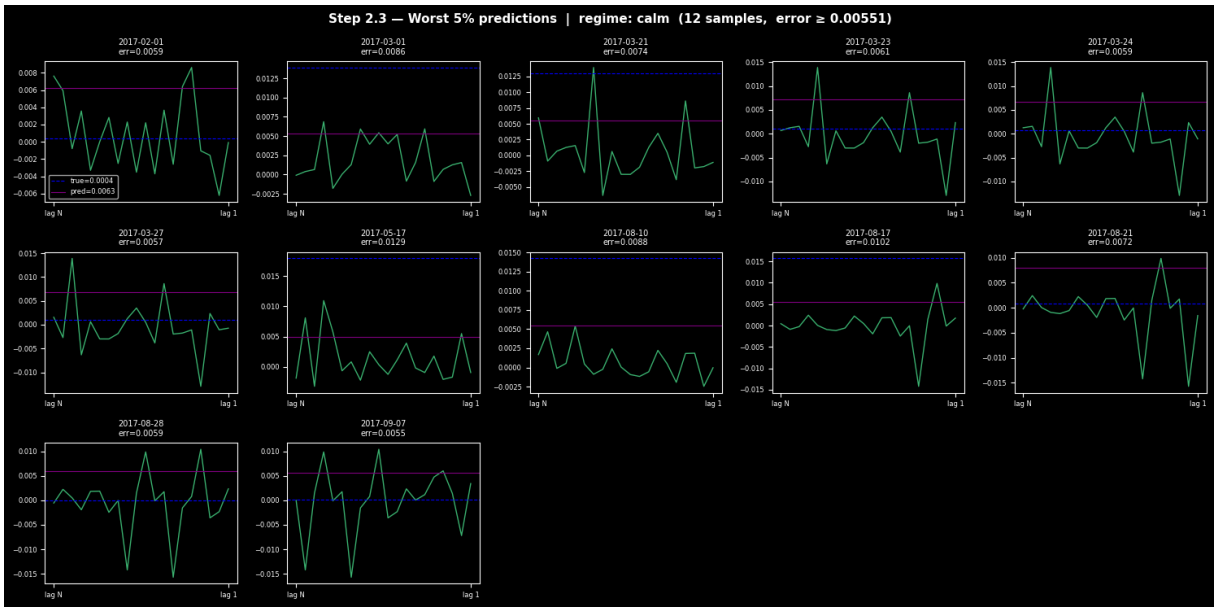

Figure 30: Worst predictions — calm regime. Failures are simply the largest true volatility values within the regime; there is no identifiable input pattern. The model's error is magnitude-driven, not structure-driven.

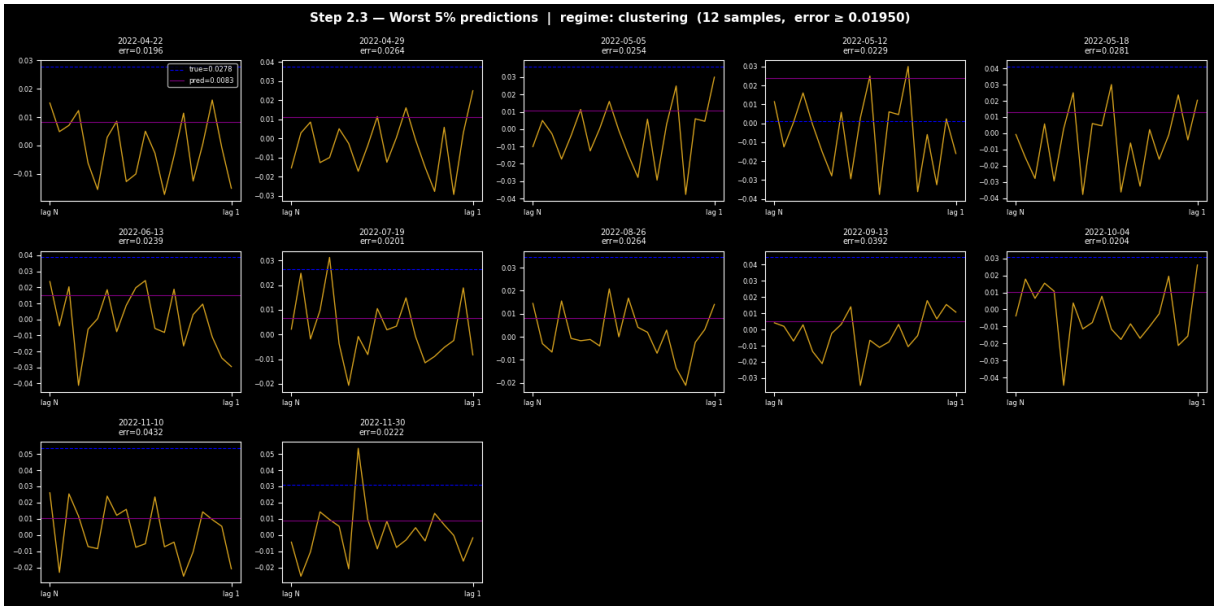

Figure 31: Worst predictions — clustering regime. Same pattern as calm: failures coincide with the largest true values; no systematic lookback pattern is visible.

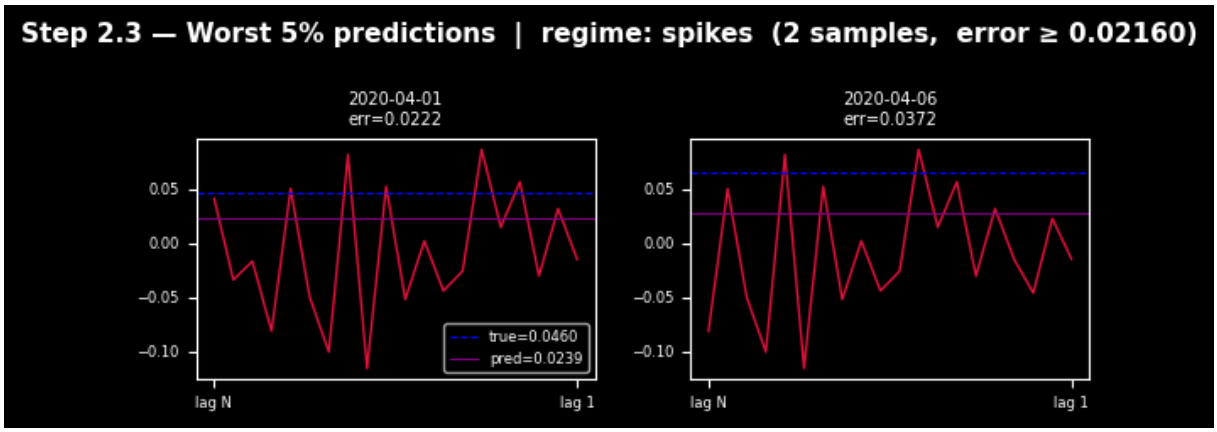

Figure 32: Worst predictions — spikes regime. Both worst cases show extreme COVID-level volatility (≈ 0.046) already in the input window. The model's prediction is far too small: it never learned to output values of this magnitude because they were essentially absent from the training set.

In the calm and clustering regimes, failures are simply the largest true values within the regime: there is no structural input pattern that a better model could exploit. In the spikes regime, however, both worst predictions show extreme COVID-level volatility already present in the lookback window, yet the model still outputs a near-mean prediction. **The failure is not a memory limitation** — the input context is already extreme. It is an *output-range problem*: the model never encountered targets of magnitude ~ 0.046 during training and therefore never learned to produce predictions in that range.

5.6 Vanishing Gradient Diagnosis

5.6.1 W_{hh} Spectral Radius

During BPTT, the gradient signal flowing back k steps is multiplied by $(W_{hh}^{\top})^k$. The fate of each gradient direction is determined by the corresponding singular value σ_i of W_{hh}: if $\sigma_i < 1$ the contribution of that direction vanishes; if $\sigma_i > 1$ it explodes.

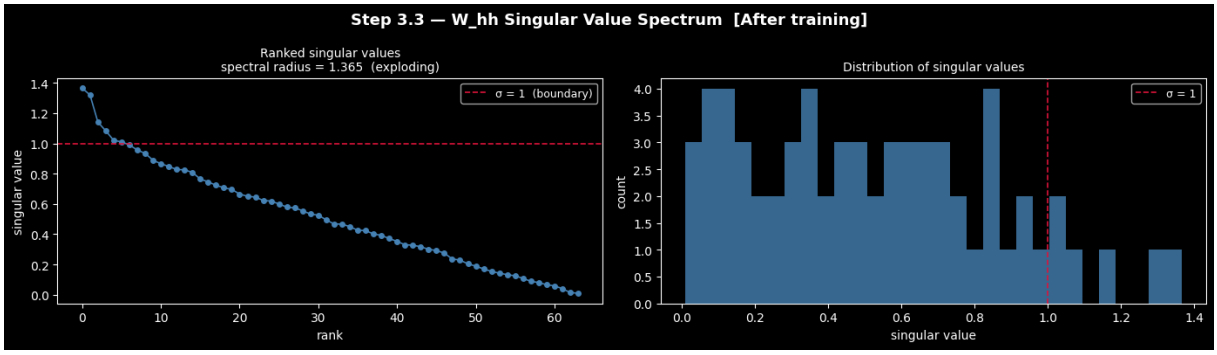

Figure 33: Singular value spectrum of W_{hh} after training. Spectral radius $\rho = 1.4$ (exploding regime overall), but roughly 57 of the 64 directions lie below 1. Training has deliberately pushed 5–7 directions above 1 to maintain gradient flow through the tanh saturation; gradient clipping prevents these from causing instability.

The spectral radius of the trained W_{hh} is $\rho = 1.4$, placing the model in the exploding-gradient regime in aggregate. However, approximately 57 of the 64 singular values lie

below 1; only 5–7 directions were deliberately pushed above 1 by training. This is the only way the model can maintain gradient flow over 20 steps of tanh saturation: a handful of amplifying directions compensate for the many vanishing ones. Gradient clipping (threshold 0.5) is therefore not merely a safety measure — it is a necessary component of stable training given this spectral structure.

5.6.2 Output Sensitivity per Time Step

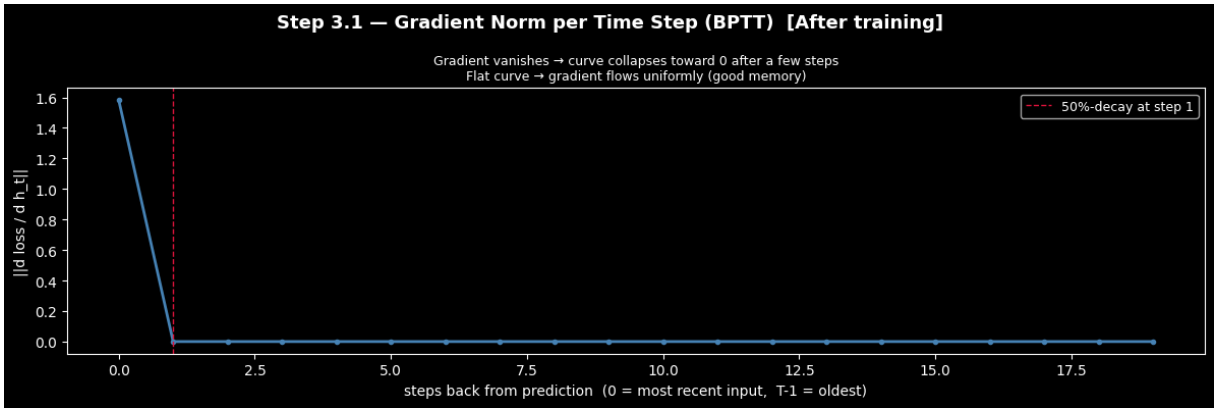

Figure 34: Output sensitivity to the hidden state at each time step: $\partial\hat{y}/\partial\mathbf{h}_t$ as a function of lag. Sensitivity decays by 50 % at step 1 and is essentially zero by step 2 — the prediction is nearly insensitive to any hidden state older than one step.

The output sensitivity to $\mathbf{h}_t$ collapses within one step: 50 %-decay occurs at lag 1 and the signal is negligible by lag 2. This is the backward view of vanishing gradients — the Jacobian of the output with respect to past hidden states decays rapidly. This seems to contradict the input sensitivity result (Section 5.4.3), which showed that perturbations to $\mathbf{x}_t$ at all lags affect the output. The next section resolves this apparent contradiction.

5.6.3 Effective Memory

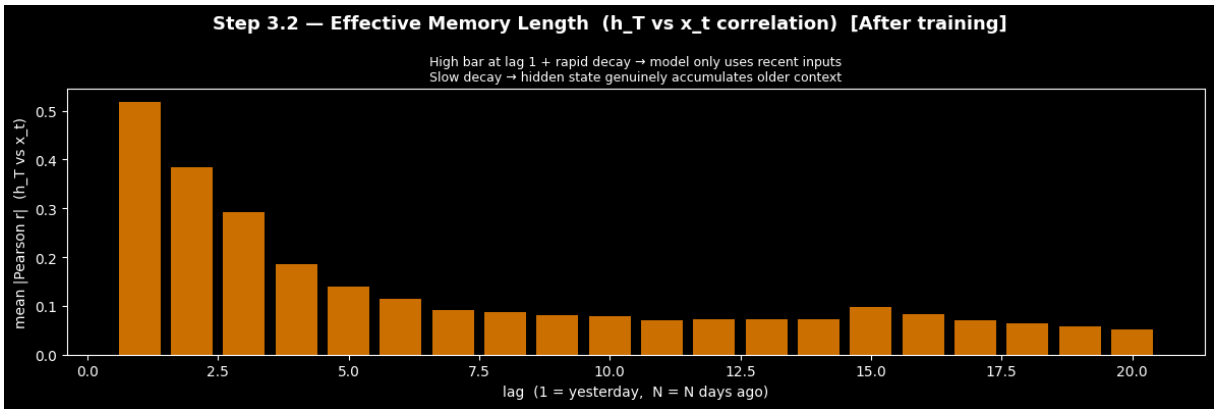

Figure 35: Effective memory: correlation between the final hidden state $\mathbf{h}_T$ and each input $\mathbf{x}_t$ at different lags. Despite the near-zero gradient norms beyond lag 2, the correlation decays gradually across the full 20-lag window and remains meaningful at lags 15–20.

The correlation between the final hidden state $\mathbf{h}_T$ and each input $\mathbf{x}_t$ decays gradually across the full 20-step window and remains non-negligible at lags 15–20. This resolves the apparent contradiction with Section 5.6.2: the two metrics measure different things.

- **Output sensitivity** (Section 5.6.2) measures the local linear derivative $\partial\hat{y}/\partial\mathbf{h}_t$: how much the output changes if the hidden state at step t is infinitesimally perturbed. This collapses because the Jacobian of the recurrence is small in most directions.
- **Effective memory** (here) measures the global nonlinear correlation between $\mathbf{h}_T$ and $\mathbf{x}_t$: whether the information from past inputs is encoded in the final state, regardless of the local derivative.

A function can be strongly correlated with a variable while having a near-zero local derivative at the current operating point. The forward dynamics of the RNN do accumulate historical context into $\mathbf{h}_T$; it is the backward gradient path that fails to propagate credit for that information.

5.6.4 Effective Memory by Regime

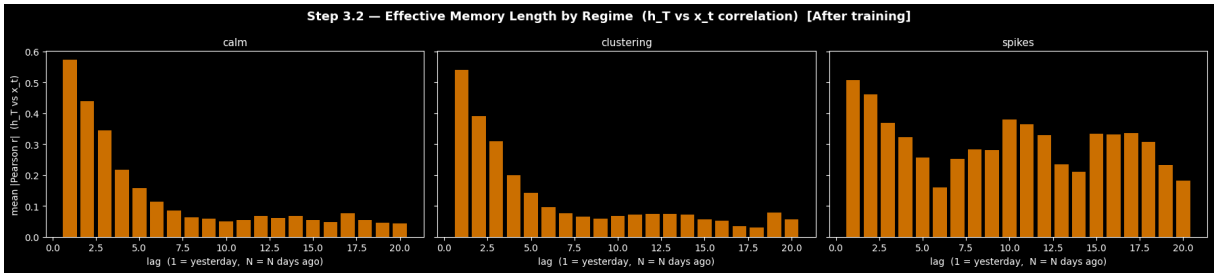

Figure 36: Effective memory split by regime. Calm: smooth exponential decay from ≈ 0.55 to 0.05–0.12. Clustering: similar shape but sharper drop, concentrated on the most recent days. Spikes: highest initial correlation (≈ 0.60) with a non-monotonic profile and secondary peaks at lags 7–11 and 12–17, reflecting the autocorrelation structure of the COVID crash itself.

The three regimes produce qualitatively different memory profiles:

- **Calm:** Smooth decay from ≈ 0.55 (lag 1) to 0.05–0.12 (lag 20). The hidden state carries a clean, recency-weighted summary of recent history.
- **Clustering:** Similar shape but a sharper initial drop; the model relies more heavily on the most recent days, consistent with the autocorrelated nature of volatility clustering.
- **Spikes:** Highest initial correlations (≈ 0.60) and a non-monotonic profile with secondary peaks at lags 7–11 and 12–17. Every day in the crash window is extreme; the secondary peaks reflect the autocorrelation structure of the crash itself, not a failure of the memory mechanism.

5.7 Hyperparameter Grid Search

A grid search over 48 configurations was run to validate the baseline architecture choices:

$$\text{seq_len} \in \{10, 20, 30\}, \quad H \in \{32, 64\}, \quad \text{layers} \in \{1, 2\},$$
$$\alpha\,(\text{learning rate}) \in \{5{\times}10^{-4},\ 10^{-3}\}, \quad \text{clip} \in \{0.5, 1.0\}.$$

Each configuration was trained with a light budget (80 epochs, early stopping at patience 15) to allow broad coverage.

Table 20: Top-5 configurations from the grid search (ranked by validation MAE). Spread from rank 1 to rank 48 is only 0.00042 in validation MAE.

Rank	seq	H	layers	α	clip	val MAE	calm	clust.	spikes
1	20	64	1	$5{\times}10^{-4}$	0.5	0.004219	0.002860	0.008079	0.009979
2	30	64	2	10^{-3}	0.5	0.004228	0.002948	0.007597	0.008271
3	20	32	2	$5{\times}10^{-4}$	1.0	0.004229	0.002763	0.007710	0.010274
4	10	64	2	10^{-3}	0.5	0.004238	0.002633	0.007917	0.015570
5	30	32	1	$5{\times}10^{-4}$	0.5	0.004242	0.003222	0.007692	0.007998

Two conclusions emerge from the grid search:

Architecture is the ceiling, not hyperparameters. The spread from rank 1 to rank 48 is only 0.00042 in validation MAE — essentially nothing. Tuning hidden size, number of layers, learning rate, or gradient clipping produces negligible gains. The model's performance ceiling is set by its architecture, not by any hyperparameter combination within this search space.

Sequence length matters, but only for the spikes regime. Sequence length 10 is consistently weak on the spikes regime (MAE = 0.015–0.017); sequence length 30 clusters around MAE $\approx$ 0.008 on spikes — a meaningful improvement. Crucially, this benefit arises from *forward-pass accumulation*, not from improved gradient-based learning: as shown in Section 5.6.2, gradients do not flow back beyond lag 2, but the forward dynamics do encode a longer history into $\mathbf{h}_T$ when more input steps are provided. The longer sequence helps because the COVID crash was a sustained event; seeing more of it in context improves the hidden state at inference time. The trade-off is a small degradation on the calm regime, where longer windows introduce additional noise from distant, uninformative lags.

5.8 Summary

Table 21: Best RNN configuration (baseline).

Parameter	Choice	Rationale
Sequence length	20	Balance between context and calm-regime noise
Hidden size	64	Sufficient capacity; larger gives no gain
Layers	1	Additional layers add complexity without benefit
Optimizer	Adam, $\alpha = 5{\times}10^{-4}$	Stable convergence
Grad. clipping	0.5	Required given $\rho(W_{hh}) = 1.4$

The RNN achieves **MAE** = 0.502 (standardised) versus **0.579** for the best FNN — a **13 %** improvement — by exploiting temporal ordering and accumulating context across the 20-step sequence. Three diagnostic findings characterise the model's behaviour and limitations:

1. **Mean-reversion bias is an output-range problem, not a context problem.** The model fails on extreme days not because it lacks historical information (the crash context is clearly encoded in the hidden state) but because it never encountered targets of that magnitude during training and therefore never learned to output them.

2. **Forward dynamics accumulate history; backward gradients do not.** The effective memory (correlation-based) spans 20 lags; the gradient-based sensitivity collapses within 2 steps. Longer sequences improve inference by enriching the forward hidden state, even though the recurrence matrix cannot propagate credit for that information backward.

3. **Architecture, not hyperparameters, is the binding constraint.** A 48-configuration grid search produces a spread of only 0.00042 MAE. The vanilla RNN is structurally limited: the final prediction depends almost entirely on $\mathbf{h}_T$, which is the hidden state most strongly influenced by the most recent step. The model sees 30 lags of history but learns primarily from the last one.

These structural limitations suggest that a gated architecture such as the LSTM is a natural next step, as its cell state is specifically designed to address the gradient collapse identified here.

6 Conclusion

6.1 Cross-Model Comparison

Table 22 summarises the performance of all models on the full test set using MAE as the primary metric. Each model is evaluated on the same held-out test period; the improvement column is computed relative to the raw linear regression baseline (MAE = 0.638).

Table 22: Cross-model comparison (MAE on test set, standardised target). Lower is better.

Model	MAE (full)	vs. linear baseline
Linear Regression (raw lags)	0.638	—
Linear Regression (feature-engineered)	0.598	−6 %
GARCH(1,1)†	0.00583	—
FNN (ReLU, $L = 3$, SGD $\mu = 0.95$)	0.579	−9 %
RNN ($T = 20$, $H = 64$, Adam)	0.502	−21 %

† GARCH is estimated on unscaled returns; its MAE is not directly comparable to the other four models.

Every neural network model outperforms the linear baseline, and the gap widens as architectural complexity increases. The RNN, which explicitly models the temporal ordering of lags, achieves the largest gain: a **21 % reduction in MAE** relative to the raw linear regression, or **13 %** relative to the best FNN configuration.

6.2 Key Findings

Three themes emerge consistently across all architectures.

Non-linearity matters, but is not enough. Moving from linear regression to an FNN reduces MAE from 0.638 to 0.579 (−9 %). The ReLU activation allows the network to learn the approximate symmetry $f(x) \approx f(-x)$ required for volatility prediction — a representation that is fundamentally unavailable to a linear model with signed inputs. Feature engineering (squared lags) partially compensates for this limitation without introducing learned non-linearity, bringing the linear baseline to 0.598. However, neither linear nor FNN models exploit the *ordering* of lags: their performance is largely insensitive to input permutation.

Temporal structure is a genuine source of signal. The RNN's 13 % improvement over the FNN demonstrates that the sequential ordering of past returns carries predictive information beyond their set-level statistics. The model learns to assign greater sensitivity to recent lags and to accumulate context across the input window — a capability unavailable to a flat feature-vector architecture. Yet the gain is partially undermined by vanishing gradients: output sensitivity collapses within approximately one recurrence step, limiting the effective memory despite a 20-lag input window.

Regime blindness is the dominant unsolved challenge. Across all architectures, crisis-period MAE is **2.5–4× higher** than calm-period MAE. During stress episodes (dot-com crash, 2008 GFC, COVID shock), every model systematically under-predicts volatility because the training distribution offers few examples of extreme moves. The ARCH signal that makes forecasting tractable during calm periods breaks down precisely when forecasting accuracy matters most — during crises. This gap is not closed by switching from linear to non-linear models, nor by introducing temporal memory: it reflects a fundamental data scarcity problem that cannot be resolved by architectural choices alone.

6.3 Limitations and Future Work

The results presented in this report point to several natural extensions.

Gated recurrent architectures. The vanilla RNN's vanishing-gradient problem is directly addressed by the Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) architectures, which introduce multiplicative gating mechanisms that decouple the gradient highway from the output projection. Based on the grid search results (Section 5), the ceiling imposed by architecture rather than hyperparameters suggests that a gated model is the single most promising next step.

Regime-aware modelling. All models in this study treat calm and crisis observations as draws from the same distribution. A natural extension is to condition the model on an estimated regime state — either via an explicit regime-switching layer, a Hidden Markov Model that labels observations before training, or a threshold variable (e.g. the VIX index) provided as an additional input feature.

Multivariate inputs. The forecasting task here relies exclusively on lagged log-returns of a single asset. Incorporating cross-asset information (implied volatility indices, credit spreads, macro factors) could provide leading indicators of regime transitions that are absent from the univariate input.

Attention mechanisms. Transformer-based architectures replace recurrence with self-attention, allowing the model to attend directly to any lag in the input window without the gradient attenuation that limits RNNs. This could address the long-range dependency problem identified in the vanishing-gradient analysis while remaining fully parallelisable during training.